Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к дипломному проекту
на тему:

ИНТЕГРИРОВАННАЯ СИСТЕМА УПРАВЛЕНИЯ ПОТОКАМИ ПОЛЬЗОВАТЕЛЬСКИХ ОБРАЩЕНИЙ

БГУИР ДП 1-40 03 01 032 ПЗ

Студент	Г. А. Левков
Руководитель	И. А. Ерофеев
Консультанты:	
от кафедры ИИТ	И. А. Ерофеев
по экономической части	Т. Л. Слюсарь
Нормоконтролёр	В. В. Захаров

Рецензент

Минск 2025

Решением рабочей комиссии
допущен(а) к защите дипломного проекта

Председатель рабочей комиссии

_____ Д. В. Шункевич
(Подпись) (Инициалы и Фамилия)

____ _____ 2025 г.

Учреждение образования
Белорусский государственный университет
информатики и радиоэлектроники

Кафедра интеллектуальных информационных технологий

УТВЕРЖДАЮ

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

___ _____ 2025 г.

ЗАДАНИЕ
на дипломный проект

Обучающемуся Левкову Глебу Александровичу
(фамилия, собственное имя, отчество (если таковое имеется))

Курс 4 Учебная группа 121703

Специальность 1-40 03 01 Искусственный интеллект

Тема дипломного проекта
Интегрированная система управления потоками пользовательских обращений
(наименование темы)

Утверждена руководителем УВО БГУИР №371-с от 10 февраля 2025 г.

Исходные данные к дипломному проекту: Каналы взаимодействия: Telegram, Web-интерфейс; OpenAI-совместимая LLM; Графовая база знаний; Язык разработки: TypeScript

Перечень подлежащих разработке вопросов или краткое содержание расчетно-пояснительной записки:
Введение
1 Анализ подходов к разработке интегрированных сиситем управления потоками пользовательских обращений
2 Проектирование интегрированной сиситемы управления потоками пользовательских обращений
3 Реализация интегрированной сиситемы управления потоками пользовательских обращений
4 Технико-экономическое обоснование реализации интегрированной сиситемы управления потоками пользовательских обращений
Заключение

Перечень графического материала (с точным указанием обязательных чертежей и графиков):
Диаграмма итеративного алгоритма поиска — формат А1, лист 1.
Диаграмма вариантов использования — формат А1, лист 1.
Диаграмма обработки запроса — формат А1, лист 1.
Диаграмма архитектуры системы — формат А1, лист 1.
Диаграмма процесса обработки обращения — формат А1, лист 1.
Диаграмма пользовательского интерфейса — формат А1, лист 1.
Диаграмма последовательности обработки обращений — формат А1, лист 1.

Консультанты по дипломному проекту (с указанием разделов, по которым они консультируют):
Раздел 1–3 — консультант от кафедры И.А. Ерофеев
Раздел 4 — консультант по экономической части Т.Л. Слюсарь

Примерный календарный график выполнения дипломного проекта

№ п/п	Наименование этапа дипломного проекта	Объем этапа, %	Срок выполнения этапа
1	Обзор литературных источников по теме, изучение проблемной области	10	25.03 – 21.04
3	Определение требований к реализации	10	30.03 – 20.04
4	Проектирование модели подсистемы	25	10.04 – 30.04
5	Разработка подсистемы поиска по графовой базе знаний	30	21.04 – 20.05
6	Экономическое обоснование принятого решения, определение экономической эффективности внедрения полученных результатов	10	01.05 – 20.05
7	Оформление расчетно-пояснительной записки	10	30.03 – 29.05
8	Оформление графического материала	5	01.05 – 29.05

Дата выдачи задания 10 февраля 2025 г.
Срок сдачи студентом законченного дипломного проекта 30 мая 2025 г.

Руководитель дипломного проекта _____ И. А. Ерофеев
(подпись) (инициалы, фамилия)

Подпись обучающегося _____

Дата ____ _____ 2025 г.

РЕФЕРАТ

РАЗРАБОТКА ИНТЕГРИРОВАННОЙ СИСТЕМЫ УПРАВЛЕНИЯ ПОТОКАМИ ПОЛЬЗОВАТЕЛЬСКИХ ОБРАЩЕНИЙ: дипломный проект/ Г. А. Левков. – Минск : БГУИР, 2025, – п.з. – 79 с., чертежей (плакатов) – 7 л. формата А1.

Целью дипломного проекта является разработка интеллектуальной системы, предназначенной для эффективного управления потоками пользовательских обращений. Система нацелена на сокращение времени обработки типовых запросов, снижение операционных затрат и повышение качества обслуживания за счет применения подхода к обработке информации, основанного на иерархической базе знаний и управляемом поиске при помощи LLM.

Предметом исследования являются методы и алгоритмы интеллектуальной обработки информации, архитектурные подходы к построению масштабируемых и адаптивных систем поддержки, а также технологии интеграции языковых моделей (LLM) для помощи в навигации по структурированным базам знаний.

Разработанная интеллектуальная система должна позволить осуществлять автоматизированную обработку пользовательских запросов, предоставляя точные и объяснимые ответы на основе верифицированной базы знаний. Ключевой особенностью является использование LLM не для генерации ответов, а в качестве интеллектуального навигатора, который интерпретирует запрос пользователя и управляет процессом последовательного углубления в иерархическую базу знаний для поиска нужного фрагмента знаний, как это мог бы делать оператор. Система должна обеспечивать прозрачность процесса принятия решений, предоставляя пользователю информацию о том, как был найден ответ.

В первом разделе пояснительной записки проведен анализ существующих подходов и коммерческих решений в области автоматизации обработки пользовательских обращений. Выявлены их ограничения, такие как недостаточная гибкость, проблема “черного ящика“ и риски, связанные с неконтролируемой генерацией ответов больших языковых моделей. Обоснована актуальность разработки собственной системы, основанной на принципах прозрачности, объяснимости и управляемости.

Во втором разделе описано проектирование интеллектуальной системы. Представлена концептуальная модель, основанная на иерархической базе знаний и роли LLM как навигатора по ней. Проработаны варианты использования системы пользователями и операторами. Разработана абстрактная модель данных для ядра системы и ее конкретизация для задачи управления обращениями пользователей. Описаны интеллектуальные механизмы обработки, включая автоматическую обработку обращений и

прогнозирование эскалаций. Также предложены архитектурные решения, обеспечивающие модульность, масштабируемость и асинхронное взаимодействие компонентов.

В третьем разделе изложены этапы практической реализации системы. Описан выбранный технологический стек, структура исходного кода. Детально рассмотрена реализация интеллектуального ядра: алгоритм последовательного поиска по графу знаний, управляемый LLM; интеграция с языковыми моделями для интерпретации запросов и навигации; механизмы формирования объяснимых ответов с указанием пути поиска в графе. Также описаны реализации адаптера для Telegram-бота и web-приложения, как примеры пользовательских интерфейсов.

В четвертом разделе приведено технико-экономическое обоснование целесообразности разработки и внедрения программного продукта, включая расчет затрат и оценку потенциального экономического эффекта.

Результатом дипломного проектирования является программный прототип интегрированной системы управления потоками пользовательских обращений, включающий универсальное ядро для поиска по иерархической базе знаний, демонстрационный Telegram-бот и web-приложение. Разработанное ядро обладает высоким потенциалом для адаптации к различным предметным областям, требующим точной и объяснимой обработки информации.

СОДЕРЖАНИЕ

Перечень условных обозначений 7
Введение 8
1 Анализ подходов к разработке интегрированной сиситемы управления потоками пользовательских обращений 10
1.1 Актуальность систем управления пользовательскими обращениями 10
1.2 Анализ существующих решений 11
1.2.1 Современные технологии и подходы 12
1.2.2 Обзор существующих решений 12
1.2.3 Вывод по анализу существующих решений 16
1.2.4 Тендецнии 18
1.2.5 Причины разработки собственного решения 18
1.3 Критерии для разработки системы 20
1.4 Вывод 22
2 Проектирование интегрированной сиситемы управления потоками пользовательских обращений 23
2.1 Концептуальная модель системы управления обращениями . 23
2.2 Варианты использования пользвателями системы 26
2.2.1 Основные сценарии работы с системой 26
2.2.2 Администрирование системы 28
2.3 Проектирование модели данных 29
2.4 Интеллектуальные механизмы обработки обращений 31
2.4.1 Автоматическая обработка обращений 34
2.4.2 Прогнозирование эскалаций 36
2.5 Архитектурные решения системы 36
2.5.1 Разделение команд и запросов (CQRS) 38
2.5.2 Изоляция главного сервиса через брокер сообщений . . 39
2.5.3 Дополнительные архитектурные компоненты системы . 39
2.6 Вывод 41
3 Реализация интегрированной сиситемы управления потоками пользовательских обращений 42
3.1 Общие сведения о реализации 42
3.2 Реализация архитектуры системы 44
3.2.1 Структура реализованных модулей и перспективы микросервисной архитектуры 45
3.2.2 Организация асинхронного взаимодействия 46
3.2.3 Реализация универсального API взаимодействия 47
3.3 Реализация подсистемы управления знаниями 47

3.3.1 Модель хранения иерархической базы знаний 47
3.3.2 Реализация хранилища контекстной информации . . . 49
3.4 Реализация ядра интеллектуальной обработки запросов . . . 49
3.4.1 Реализация алгоритма последовательного интеллектуального поиска 49
3.4.2 Интеграция с языковыми моделями (LLM) 50
3.4.3 Формирование и объяснение ответов 51
3.4.4 Реализация интеллектуальных механизмов автоматизации 52
3.5 Реализация пользовательских интерфейсов (Адаптеры) . . . 53
3.5.1 Реализация адаптера для Telegram-бота 54
3.5.2 Реализация web-приложения 55
3.6 Результаты разработки и демонстрация работы 61
3.7 Вывод по разделу 63
4 Экономическое обоснование разработки системы мониторинга и управления производственными линиями 64
4.1 Характеристика программного средства 64
4.2 Расчет затрат на разработку и определения цены программного средства 65
4.2.1 Расчет затрат на основную заработную плату разработчикам 65
4.2.2 Расчет затрат на дополнительную заработную плату разработчикам 66
4.2.3 Расчет отчислений на социальные нужды. 66
4.2.4 Расчет затрат на прочие расходы 67
4.2.5 Расчет суммы затрат на разработку 67
4.2.6 Результаты расчета цены на разработку программного средства 68
4.3 Расчет результата от разработки и реализации программного средства 68
4.4 Расчет показателей экономической эффективности разработки и использования программного средства в организации . 70
4.5 Вывод 71
Заключение 74

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В пояснительной записке используются следующие условные обозначения:
AI - Искусственный интеллект (Artificial Intelligence);
API — Программный интерфейс приложения (Application Programming Interface);
CQRS — Разделение ответственности команд и запросов (Command and Query Responsibility Segregation);
CRM - Управление взаимоотношениями с клиентами (Customer Relationship Management);
CSS — Каскадные таблицы стилей (Cascading Style Sheets);
IT — Информационные технологии (Information Technology);
JSON — Нотация объектов JavaScript (JavaScript Object Notation);
LLM - Большая языковая модель (Large Language Model);
ML - Машинное обучение (Machine Learning);
MVP — Минимально жизнеспособный продукт (Minimum Viable Product);
NLP — Обработка естественного языка (Natural Language Processing);
RAG — Генерация с дополненным поиском (Retrieval-Augmented Generation);
UML — Унифицированный язык моделирования (Unified Modeling Language);
URL — Унифицированный указатель ресурса (Uniform Resource Locator);
WS-API — Программный интерфейс веб-сокетов (Web Socket Application Programming Interface);
БЗ — База знаний;
ИИ — Искусственный интеллект;
СУБД — Система управления базами данных.

ВВЕДЕНИЕ

В современных условиях цифровой трансформации бизнеса эффективное управление коммуникациями с пользователями становится критическим фактором успеха для организаций любого масштаба. Стремительный рост объемов пользовательских обращений, увеличение числа каналов взаимодействия и повышение ожиданий клиентов относительно скорости и качества обслуживания создают новые вызовы для традиционных систем поддержки. По данным исследований, более 70% пользователей ожидают получить ответ на свое обращение в течение первых 30 минут, при этом около 40% запросов являются типовыми и повторяющимися [1]. В контексте стремительного развития и появления новых, иногда неожиданных, возможностей у больших языковых моделей (LLM) [2], традиционные подходы к обработке запросов требуют переосмысления.

Существующие решения в области управления обращениями зачастую характеризуются фрагментарностью, низкой адаптивностью к изменяющимся бизнес-процессам и недостаточной прозрачностью принятия решений. Большинство систем либо предлагают ограниченную автоматизацию простейших сценариев, либо работают как “черные ящики“ [3], не предоставляя объяснений предлагаемых решений. Это приводит к высокой нагрузке на операторов, длительному времени ожидания ответа и, как следствие, снижению удовлетворенности пользователей.

Целью данного дипломного проекта является разработка интегрированной системы управления потоками пользовательских обращений, способной сократить время обработки типовых запросов, снизить операционные затраты и повысить качество обслуживания. В основу проектируемой системы положены принципы иерархического структурирования знаний, прозрачности и объяснимости принимаемых решений, а также возможности адаптации к различным предметным областям.

В рамках дипломного проекта решаются следующие задачи:

– анализ существующих подходов к автоматизации обработки пользовательских обращений и выявление их ограничений;

– проектирование архитектуры системы, обеспечивающей возможность в автоматическом режиме отвечать на пользовательские обращений предоставляя информацию о том как было достигнуто это решение;

– разработка интеллектуальных механизмов обработки запросов, в частности, алгоритм последовательного поиска по иерархической базе знаний;

– реализация прототипа системы с возможностью интеграции с различными каналами коммуникации;

– экономическое обоснование эффективности внедрения разработанного решения.

Предлагаемый подход отличается от типовых решений тем, что система не генерирует ответы на основе языковых моделей, а осуществляет навигацию по предварительно верифицированным знаниям с обязательным указанием пути, по которому был получен ответ. Это обеспечивает прозрачность процесса и позволяет пользователю понять, почему система предложила именно такое решение.

Практическая значимость работы заключается в создании универсальной платформы для автоматизации процессов обработки обращений, которая может быть адаптирована к различным предметным областям – от технической поддержки до медицинской диагностики и юридического консультирования.

Дипломный проект выполнен самостоятельно, проверен в системе “Антиплагиат“. Процент оригинальности соответствует норме, установленной кафедрой ИИТ. Цитирования обозначены ссылками на публикации, указанные в “Списке использованных источников“.

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕГРИРОВАННОЙ СИСИТЕМЫ УПРАВЛЕНИЯ ПОТОКАМИ ПОЛЬЗОВАТЕЛЬСКИХ ОБРАЩЕНИЙ

Информационные технологии сегодня становятся определяющим фактором эффективности организационных коммуникаций. В условиях постоянно усложняющихся бизнес-процессов и растущих пользовательских ожиданий системы управления обращениями превращаются из вспомогательного инструмента в стратегический ресурс компании. Цифровая трансформация требует принципиально новых подходов к коммуникациям, где на первый план выходят гибкость, интеллектуальность и способность системы адаптироваться к меняющимся условиям.

1.1 Актуальность систем управления пользовательскими обращениями

В современном цифровом мире коммуникация между организациями и пользователями приобретает принципиально новое значение. Стремительное развитие информационных технологий, появление множества каналов взаимодействия и постоянно растущие требования к качеству сервиса создают потребность в системах способоных автоматически обрабатывать пользовательские обращения.

Традиционные подходы к обработке обращений часто страдают от фрагментарности и низкой эффективности. Это проявляется в хаотичном распределении запросов, отсутствии стандартов и долгом времени ответа (рисунок 1.1). В результате ухудшается коммуникация, что ведет к потере клиентов, снижению лояльности и репутационным потерям.



Рисунок 1.1 – Хаотичные распределение входящих запросов

Системы управления обращениями принципиально меняют подход к организации коммуникативных процессов. Они позволяют создавать гибкие, настраиваемые модели взаимодействия, которые могут адаптироваться под специфику конкретной организации. Возможность динамической настройки статусных моделей, маршрутизации и типов обращений превращает такие системы в мощный инструмент оптимизации бизнес-процессов.

Особую значимость приобретает возможность автоматизации и интеллектуализации процессов обработки обращений. Современные технологии машинного обучения и искусственного интеллекта позволяют не только фиксировать и передавать обращения, но и осуществлять их классификацию, прогнозирование и маршрутизацию. Это сокращает время реагирования на обращения, повышает точность и эффективность работы сотрудников.

Тенденция к растущей автоматизации обработки обращений становится ключевым фактором эволюции современных систем поддержки пользователей. В рамках разрабатываемой системы предлагается подход, позволяющий обеспечить функционирование системы в автономном режиме с минимальным вмешательством человека-оператора. Техническое решение базируется на структурированной базе знаний в виде графа декомпозиции, где каждый узел содержит определенное знание, а связи между узлами формируют многоуровневую структуру детализации информации. Аналитический модуль при поступлении обращения автоматически анализирует запрос, ищет проблему и осуществляет поиск подходящих решений в графе знаний, что в теории радикально сокращает время обработки и повышает качество обслуживания. Данный механизм представляет собой естественное развитие концепции систем управления обращениями в условиях цифровой трансформации бизнес-процессов.

Экономический эффект от внедрения подобных систем также положителен. Автоматизация рутинных процессов, сокращение операционных издержек, повышение производительности сотрудников и улучшение качества сервиса создают явные конкурентные преимущества для организаций.

Таким образом, актуальность разработки интеллектуальных систем управления пользовательскими обращениями обусловлена комплексом технологических, экономических и организационных факторов. Такие системы становятся не просто инструментом коммуникации, но стратегическим ресурсом, способным существенно повысить эффективность взаимодействия организации с её пользователями.

1.2 Анализ существующих решений

В условиях растущих требований пользователей к качеству сервисов всё более актуальной становится задача эффективной обработки обращений. Организации стремятся минимизировать время отклика, снизить нагрузку

на сотрудников поддержки и при этом обеспечить высокий уровень персонализации своей системы. Это стимулирует развитие решений, способных автоматизировать и интеллектуализировать процессы коммуникации с пользователями. Рассмотрим современные подходы и существующие разработки в этой области на данный момент.

1.2.1 Современные технологии и подходы

На сегодняшний день интеллектуальные системы обработки обращений уже не являются экспериментальными решениями, а активно внедряются в крупные компании. Основной фокус смещается в сторону автоматизации рутинных операций, интеллектуальной маршрутизации и анализа запросов с использованием методов машинного обучения и обработки естественного языка (NLP). Наиболее популярные решения включают: использование ML-моделей для классификации обращений, динамическое распределение нагрузки между операторами, а также интеграцию с базами знаний для предложений готовых ответов.

Появление так называемых “AI-агентов“, способных обрабатывать такие обращения и самостоятельно вести диалог с пользователями, стало ключевым трендом последних лет, рисунок 1.2. Однако несмотря на стремительный прогресс, большинство существующих решений сталкиваются с ограничениями в адаптивности, качестве интерпретации запросов и глубине интеграции с корпоративными знаниями.

1.2.2 Обзор существующих решений

Среди наиболее заметных решений в области автоматизации обработки обращений стоит выделить Zendesk Resolution Platform[4], представленную в 2025 году. Это комплексная платформа, ориентированная на использование агентного искусственного интеллекта на базе больших языковых моделей для автоматизации пользовательских взаимодействий. Платформа предоставляет обширный инструментарий для построения и обучения AI-агентов, работающих на основе корпоративной базы знаний. К её сильным сторонам относятся высокая гибкость, масштабируемость и развитые возможности для интеграций с другими решениями. Однако, несмотря на свою технологическую мощь, платформа остаётся довольно закрытой, требует значительных ресурсов на внедрение и плохо адаптируется под нестандартные сценарии взаимодействия.

Ещё одним примером современного подхода является система Intercom Fin, представляющая собой интеллектуального AI-агента, специально созданного для обслуживания пользователей в различных каналах коммуникации, рисунок 1.3. Ключевая особенность Intercom Fin — способность к

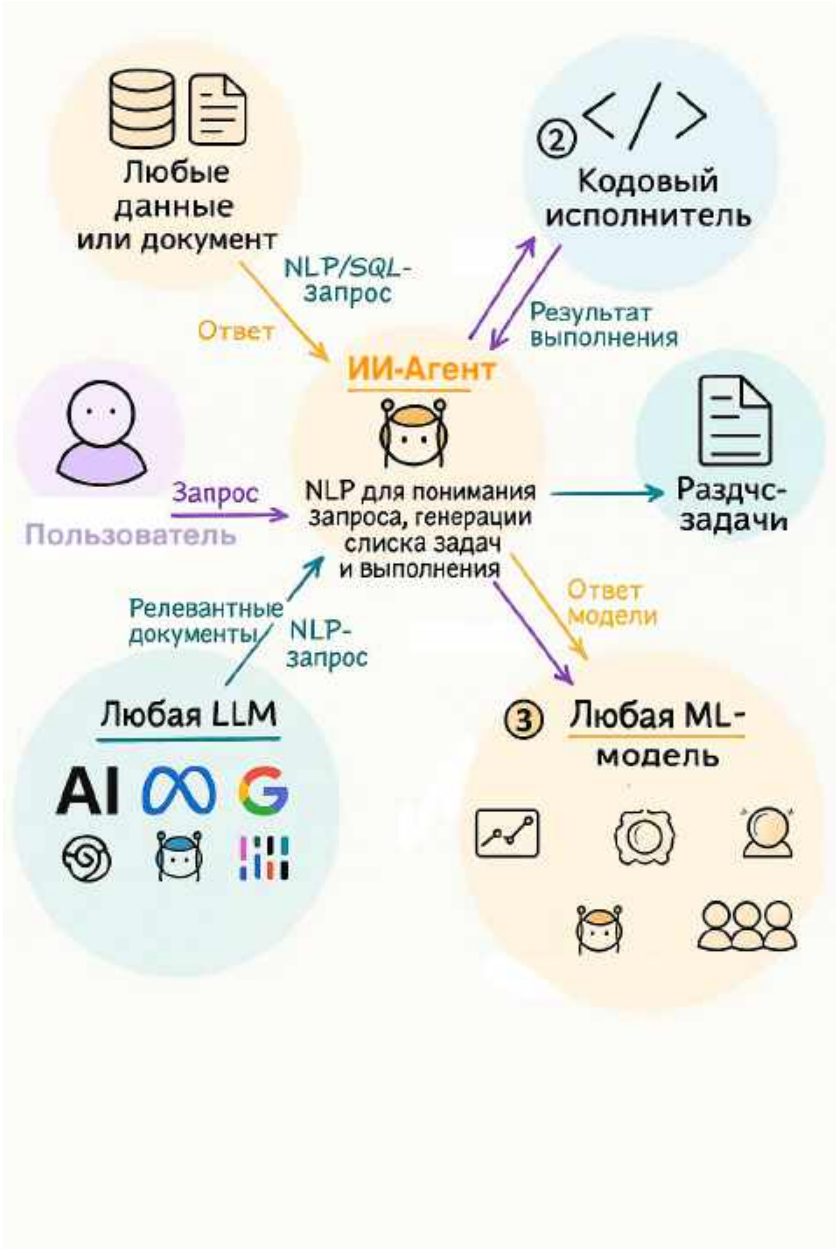


Рисунок 1.2 – AI-агент

быстрой адаптации и обучению в ходе реальных диалогов. Разработчики активно развивают самообучающиеся алгоритмы, делая упор на персонализацию взаимодействия. Однако при этом система имеет ограниченные возможности по глубокой маршрутизации и не поддерживает сложные многоуровневые структуры знаний, что может стать сдерживающим фактором в условиях комплексных бизнес-процессов.

Существенно иным подходом отличается решение SentiSum[5], ориентированное на автоматическое распознавание тематики и эмоционального окраса обращений. Основная задача этой системы — интеллектуальная маршрутизация запросов по релевантным направлениям. За счёт использо-



Рисунок 1.3 – Intercom Fin AI-агент

вания NLP-моделей, система способна эффективно распределять обращения между командами, учитывая их специфику. Она отличается высокой степенью возможностей интеграции с популярными CRM-сервисами. Тем не менее, ограниченная настраиваемость моделей и низкая прозрачность алгоритмов принятия решений создают сложности в управлении и интерпретации поведения системы со стороны операторов.

Система IrisAgent представляет собой продвинутый инструмент интеллектуальной маршрутизации обращений, в которой учитываются не только содержательная часть запроса, но и такие факторы, как загруженность операторов и соответствие их компетенций. Подобный подход особенно полезен для крупных организаций с распределёнными командами поддержки. Преимуществом является способность адаптироваться под внутренние метрики эффективности, однако система требует высокой точности и полноты входных данных о персонале и может быть сложной в конфигурации и сопровождении, рисунок 1.4.

Интересный и более лёгкий в освоении подход демонстрирует платформа Labelf AI[6], которая делает ставку на быстрое обучение кастомных моделей на основе обращений конкретной компании. Благодаря удобному интерфейсу и поддержке различных языков, система может быть быстро внедрена в бизнес-процессы без необходимости длительной подготовки данных. Однако в случае масштабных или многоуровневых сценариев взаимодействия Labelf AI может оказаться менее эффективной — её слабые стороны проявляются в ограниченной аналитике и трудностях масштабирования.

Помимо платформ, ориентированных на общую автоматизацию кли-

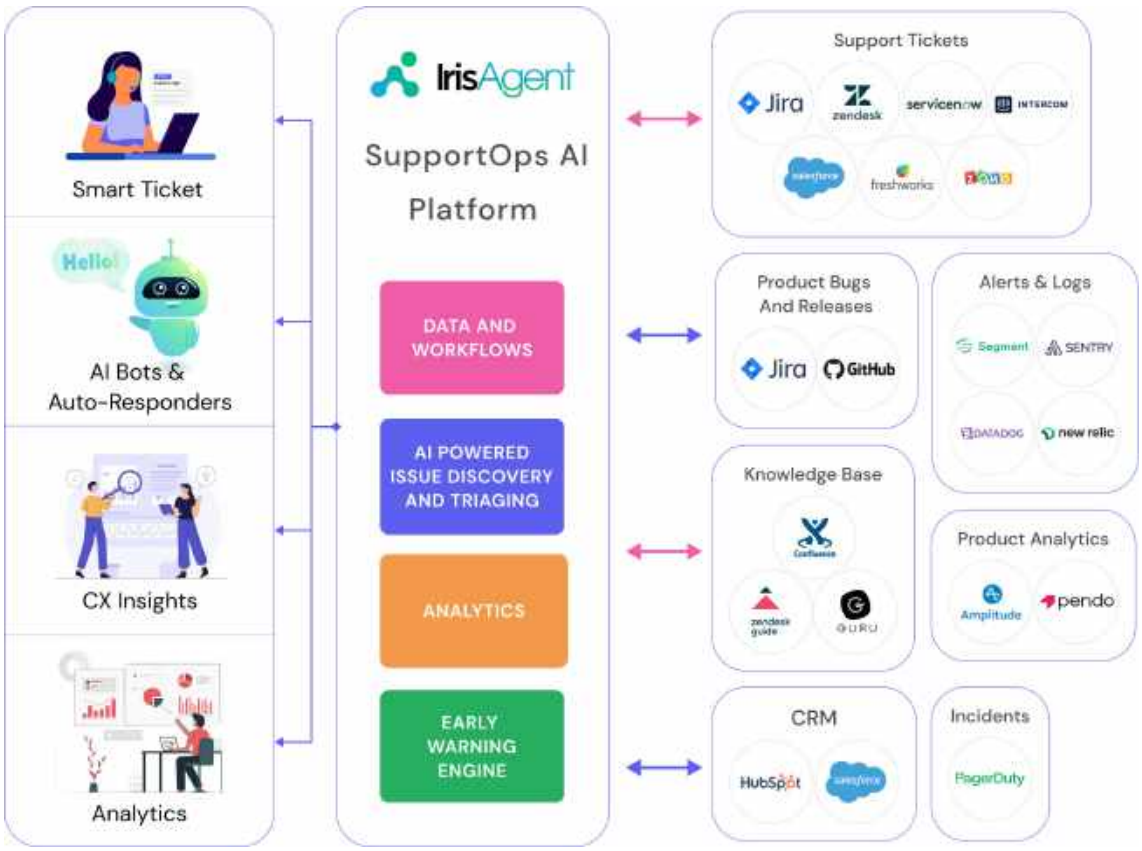


Рисунок 1.4 – IrisAgent AI-агент

ентского сервиса, существуют и более специализированные системы, использующие структурированные базы знаний и элементы искусственного интеллекта для решения сложных задач в корпоративном секторе.

IBM Watson[7] представляет собой платформу с набором сервисов искусственного интеллекта, которые могут быть применены в различных отраслях, таких, например, как здравоохранение. Примером может быть Watson for Oncology[8], который был разработан для помощи в выборе методов лечения рака. Пример схемы его работы можно увидеть на рисунке 1.5. Ключевой особенностью Watson является его способность обрабатывать и анализировать большие объемы неструктурированных и структурированных данных (научные статьи, медицинские записи, финансовые отчеты), понимать запросы на естественном языке и предоставлять ответы, подкрепленные доказательствами из исходных данных. Хотя IBM Watson является довольон специфичным, комплексным и ресурсоемким решением, его архитектура, основанная на глубоком анализе верифицированных источников знаний для предоставления точных ответов, перекликается с идеей использования управляемой базы знаний для повышения надежности и релевантности ответов в системах поддержки. К минусам можно отнести сложность в его использование обусловаленую его аппаратной сложностью

и специфичностью предметной области.

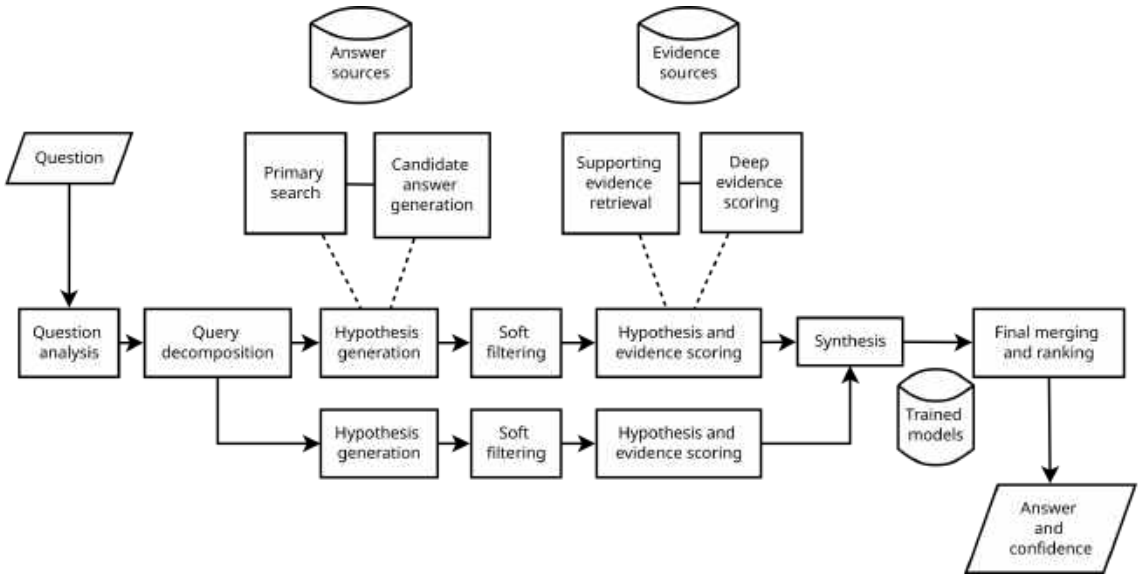


Рисунок 1.5 – Схема работы IBM Watson

Другим примером использования подобного подхода в связке с графовой базой знаний является SAP Datasphere Knowledge Graph[9], рисунок 1.6. Этот компонент SAP предназначен для структурирования “озёр данных“ (data lakehouse) SAP. Его основная задача – выявлять и представлять связи между различными сущностями данных и бизнес-процессами из разнообразных источников. SAP Knowledge Graph создает семантический слой поверх данных, делая их более понятными и доступными для анализа и использования, особенно для генеративных моделей ИИ. Он позиционируется как фундаментальный элемент для таких инструментов, как встроенного в SAP Copilot, позволяя улучшить качество, релевантность и объяснимость ответов, генерируемых ИИ, за счет предоставления сложных взаимосвязей в бизнес-данных в структурированном виде. Граф знаний в этом контексте собирает и объединяет информацию, строя модель, где узлы – это сущности (например, клиент, продукт, заказ), а ребра – это отношения между ними. Генеративные ИИ являются потребителями этого графа, используя его для получения необходимого контекста при формировании ответов на запросы пользователей. Этот механизм неотрывно связан с самим SAP, поэтому его исопльзование в других независимых от SAP систем не представляется возможным.

1.2.3 Вывод по анализу существующих решений

Проведенный анализ существующих решений на рынке систем управления пользовательскими обращениями показывает активное развитие данной области. Предлагается широкий спектр инструментов: от комплексных

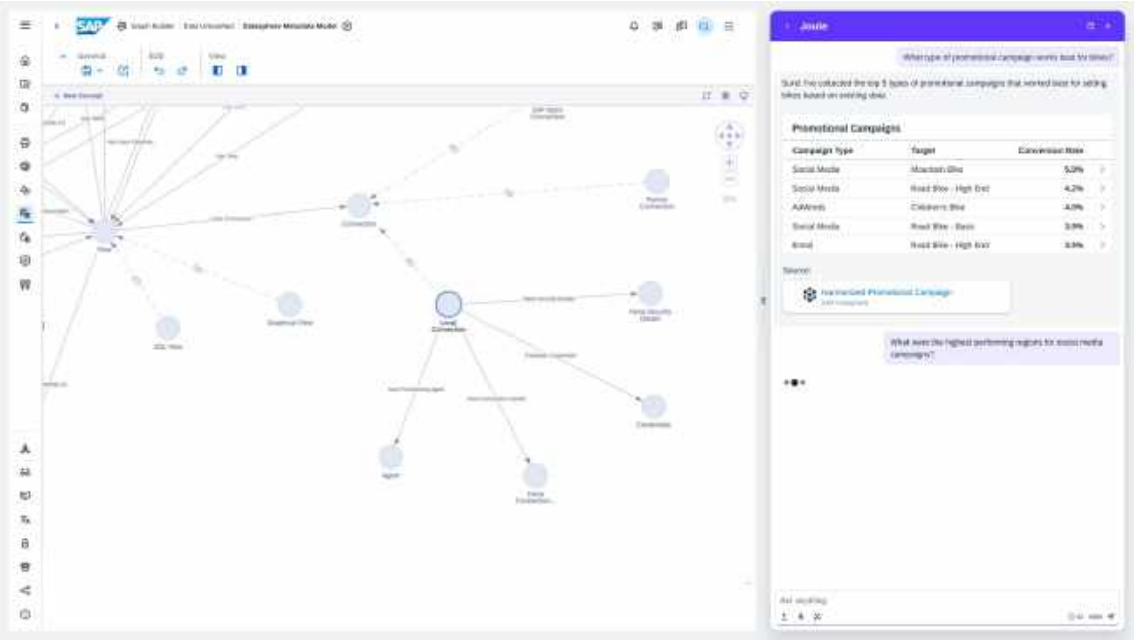


Рисунок 1.6 – Автоматически сгенерированный граф знаний в SAP

платформ типа Zendesk Resolution Platform и Intercom Fin, стремящихся к полной автоматизации взаимодействия с клиентами с помощью AI-агентов, до более специализированных решений, таких как SentiSum или IrisAgent, фокусирующихся на интеллектуальной маршрутизации, анализе тональности и предиктивной аналитике. Платформы, позволяющие быстро обучать кастомные модели на основе обращений конкретной компании, как Labelf AI, также находят свое применение, предлагая более легкий вход для внедрения ИИ.

Однако, несмотря на значительный прогресс, многие коммерческие продукты либо демонстрируют ограниченную адаптивность к глубоко специфическим базам знаний и сложным бизнес-процессам, либо функционируют как «чёрные ящики», не обеспечивая должной прозрачности и контроля над процессом формирования ответов. Это может приводить к сложностям в интерпретации поведения системы и рискам, связанным с генерацией неточной или нерелевантной информации, особенно при использовании LLM без достаточного контекстуального контроля.

В то же время, опыт промышленных гигантов, таких как IBM с платформой Watson и SAP с Datasphere Knowledge Graph, демонстрирует неоспоримую ценность использования структурированных баз знаний и графовых моделей для работы с большими объемами информации и предоставления контекстуально-обоснованных данных. IBM Watson подчеркивает важность верифицированных источников для генерации точных ответов в критических областях. SAP Knowledge Graph, в свою очередь, служит мощным семантическим слоем для корпоративных данных, предоставляя

необходимый контекст для генеративных ИИ (например, их копайлота Joule), которые уже на его основе формируют конечные ответы для пользователей. В этих масштабных системах граф знаний является либо источником для глубокого аналитического вывода, либо фундаментальной контекстной основой для последующей работы генеративных моделей.

В данном дипломном проекте предлагается использовать LLM не для генерации ответов, а как интеллектуальный инструмент поиска по заранее формализованной, иерархической и верифицированной базе знаний. Ответ формируется непосредственно из найденного узла этой базы, что обеспечивает высокую точность, контроль над содержанием и объяснимость решения. Такой подход минимизирует риски "галлюцинаций" LLM и особенно актуален для задач, требующих высокой достоверности информации и прозрачности в специфических, глубоко структурированных областях.

1.2.4 Тендецнии

Современные исследования сосредоточены на более глубокой интеграции AI с корпоративными знаниями. Одним из ключевых направлений является использование Retrieval-Augmented Generation (RAG) в связке с графовыми базами знаний [10]. Такой подход, часто именуемый GraphRAG, позволяет преодолеть ограничения традиционных LLM, такие как генерация неточной информации ("галлюцинации"), за счет предоставления моделям доступа к проверенным и структурированным данным из графа знаний. Это особенно важно для предметных областей, требующих высокой точности и объяснимости ответов, как например в этой статье[11] рассматривается использовани RAG вместе с графом знаний. В исследовании 2024 года, реализованном в LinkedIn, было показано, что применение RAG в связке с многоуровневыми структурами знаний позволило улучшить точность обработки обращений и сократить время ответа на 28,6%. Более того, использование графовых структур в RAG позволяет моделям лучше улавливать сложные контекстуальные связи между фрагментами информации, что превосходит возможности RAG, основанного исключительно на векторном поиске по неструктурированным текстам[12]. Подобные исследования демонстрируют явный сдвиг от простых чат-ботов к более сложным системам интеллектуальной поддержки, способным "понимать" контекст и структуру обращений.

1.2.5 Причины разработки собственного решения

При всех достоинствах существующих решений остаются нерешённые задачи, особенно в условиях высокой нагрузки, сложности предметной области и необходимости прозрачности принятия решений. Типичная ситуация

в крупной технологической компании иллюстрирует актуальность этих проблем: при ежедневной нагрузке в сотни обращений, большинство из которых являются повторяющимися, время ожидания первого ответа может достигать нескольких часов, а текучка персонала и затраты на обучение новых сотрудников постоянно растут.

Существующие платформы, такие как Zendesk, Intercom, или IrisAgent, фокусируются преимущественно на горизонтальной маршрутизации и автоматизации простых сценариев, редко предлагая глубоко настраиваемые архитектуры под специфику конкретного бизнеса. Они, как правило, построены вокруг уже существующих CRM-экосистем и затрудняют внедрение нестандартных моделей обработки знаний. Более того, большинству из них недостаёт прозрачности — как в плане интерпретации принимаемых системой решений, так и в возможности отслеживания пути к найденному ответу. Это критично в среде, где пользователь ожидает не просто результата, но и объяснения, почему именно такой ответ был предложен.

В ответ на эти вызовы предлагается разработать собственное решение обработки пользовательских обращений. В её основе лежит выше описанный принцип иерархического структурирования знаний и последовательного уточняющего поиска. Важность такого подхода, ориентированного на повторное использование проверенных знаний и четкую логику, подтверждается исследованиями в области управления ИТ-услугами, где подчеркивается, что своевременность и качество решения запросов напрямую зависят от возможности быстрого доступа к релевантной информации и минимизации времени на диагностику [13]. Вместо поверхностной классификации, как это часто реализовано в современных системах, предлагает навигацию по дереву знаний с возможностью уточнения запроса, выбором релевантной ветви и предоставлением обоснованного ответа. Таким образом, система имитирует логическое мышление специалиста первой линии поддержки, но делает это значительно быстрее и без риска ошибки из-за человеческого фактора.

Одним из ключевых отличий разрабатываемого решения является отказ от генерации ответов на основе языковых моделей в пользу поиска и составления ответа исключительно на базе существующих знаний. LLM используется лишь как вспомогательный инструмент — для интерпретации намерений пользователя, оценки полноты информации и навигации по графу знаний. Такой подход снижает вероятность фактических ошибок и обеспечивает самое главное - прозрачность, пользователь может увидеть, откуда получен ответ, и проследить путь к нему.

Кроме того, система предполагает возможность эскалации запроса только в случае отсутствия релевантного решения в базе знаний. При этом обращение уже будет дополнено результатами предварительного анализа и уточнениями, что существенно экономит время специалистов второй линии

и повышает общую эффективность обработки обращений.

В перспективе разрабатываемая система может быть трансформирована в платформенное решение, пригодное для применения в различных предметных областях — от технической поддержки до медицинской диагностики и юридического консультирования. Универсальность обеспечивается за счёт абстрагированной архитектуры: иерархическая база знаний, алгоритм последовательного поиска, механизм объяснения и возможности контекстных действий (например, обращения к внешним системам или автоматического запуска бизнес-процессов). Таким обоазом это не просто система поддержки, а в полноценный интеллектуальный агент, способный адаптироваться к различным условиям и задачам.

Таким образом, необходимость создания собственного решения обусловлена как практическими ограничениями существующих платформ, так и стремлением к более глубокому, объяснимому и настраиваемому подходу к интеллектуальной автоматизации. Эта азработка позволяет не только решать актуальные бизнес-проблемы, но и формирует задел для создания нового класса адаптивных систем поддержки принятия решений.

1.3 Критерии для разработки системы

Создание интеллектуальной системы обработки обращений требует не только выбора современных технологий, но и чёткого определения принципов, на которых она будет строиться. Основной вызов заключается не столько в автоматизации, сколько в построении адаптивной, объяснимой и масштабируемой архитектуры, способной эффективно функционировать в условиях постоянных изменений бизнес-процессов и роста пользовательской нагрузки.

В предлагаемой концепции, ключевыми можно выделить следующие критерии:

1. В отличие от типовых AI-решений, работающих как «чёрный ящик», система должна опираться на прозрачный механизм поиска по структуре знаний. Каждое решение должно быть обосновано, сопровождено ссылкой на источник и маршрутом в дереве знаний, рисунок 1.7. Такой подход — залог доверия и контроля, особенно в средах с высоким уровнем ответственности.

2. Роль языковых моделей должна ограничиваться интерпретацией запросов, навигацией по базе знаний и управлением диалогом с пользователем. Финальные ответы — только на основе верифицированных данных. Это исключает галлюцинации и повышает качество и предсказуемость системы.

3. Система изначально должна быть рассчитана на горизонтальное масштабирование, включая использование распределённых баз знаний, кэширования результатов поиска, асинхронной обработки обращений и балан-

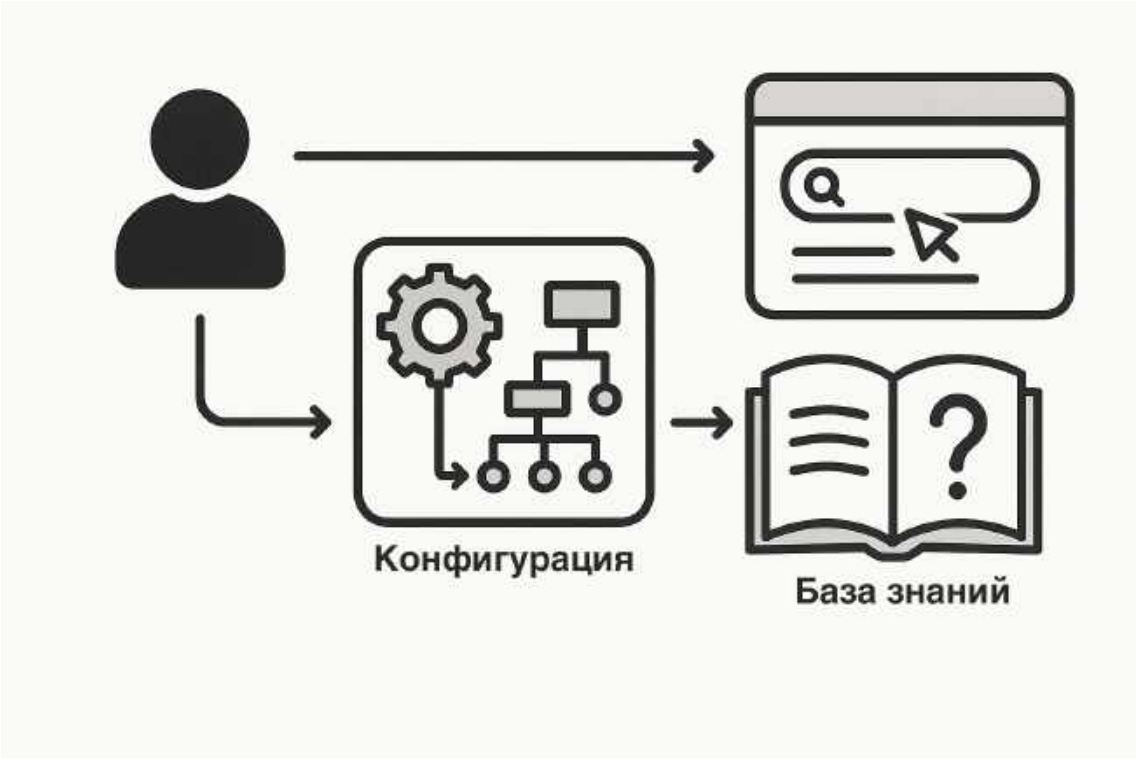


Рисунок 1.7 – Решение должно быть обоснованно

сировки нагрузки.

4. Предлагаемая модель не должна ограничиваться конкретной сферой применения. Иерархическая структура знаний и механизм последовательного уточнения запроса могут быть применимы к самым разным областям — от медицины до права. Поэтому архитектура должна быть абстрагирована от предметной области и легко адаптируема под новые задачи путём замены базы знаний и настройки логики поиска.

5. В критически важных сценариях система должна уметь объяснять, почему был предложен тот или иной ответ, какие данные были использованы, и каков был путь анализа. Это открывает возможности для аудита, отладки и доверия пользователей — как конечных клиентов, так и администраторов.

6. Ядро системы (модуль поиска решений) должно быть максимально простым, лёгким в интеграции и обеспечивать быстрое развёртывание. Архитектура должна минимизировать сложность базового функционала, позволяя запускать систему в короткие сроки, даже с минимальной конфигурацией. При этом ключевые механизмы — обработка запросов, навигация по базе знаний и формирование ответов — должны оставаться независимы от внешних зависимостей, чтобы сохранить гибкость для дальнейшего развития.

Формулируемые критерии показывают, что предложенная система — это не просто автоматизированный справочник или чат-бот, а интеллекту-

альная платформа. Только на стыке гибкости, прозрачности, масштабируемости и объяснимого AI может быть построено по-настоящему эффективное решение, способное заменить человеческого оператора там, где ранее это казалось невозможным.

1.4 Вывод

Рынок интеллектуальных решений для обработки пользовательских обращений находится на стадии активного роста и трансформации. Несмотря на наличие зрелых коммерческих продуктов, большинство из них демонстрируют либо ограниченную адаптивность, либо работают как «чёрные ящики», не предоставляя прозрачности и объяснимости при принятии решений. Для крупных компаний, сталкивающихся с большими объёмами повторяющихся запросов, такие решения либо переизбыточны, либо, наоборот, не покрывают специфические бизнес-потребности.

Предложенная концепция пытается ответить на этот вызов. Она основана на трёх ключевых идеях: иерархической структуре знаний, управляемом интеллектуальном поиске с поэтапным уточнением и объяснимой архитектуре ответов. Такой подход позволяет не просто автоматизировать обращения, а трансформировать сам принцип взаимодействия с пользователями — от реактивной поддержки к проактивному и интеллектуальному диалогу.

Разработка собственного решения становится не только обоснованным шагом с точки зрения гибкости и качества обслуживания, но и стратегической инвестицией в платформу, способную масштабироваться, переиспользоваться в других бизнес-доменах и интегрироваться в цифровую экосистему организации. В долгосрочной перспективе может стать ядром интеллектуальной поддержки принятия решений в самых разных сферах — от технической поддержки до медицины, юриспруденции и образования.

2 ПРОЕКТИРОВАНИЕ ИНТЕГРИРОВАННОЙ СИСИТЕМЫ УПРАВЛЕНИЯ ПОТОКАМИ ПОЛЬЗОВАТЕЛЬСКИХ ОБРАЩЕНИЙ

В современных условиях ведения бизнеса эффективность обработки пользовательских обращений становится одним из ключевых факторов, определяющих качество клиентского сервиса и общую операционную эффективность. Проектируемая интеллектуальная система управления потоками пользовательских обращений призвана решить комплекс проблем, связанных с большими объемами однотипных запросов, длительным временем ожидания первого ответа и высокой текучестью кадров среди специалистов первой линии поддержки. Основная цель системы — максимально сократить время обработки типовых обращений при одновременном снижении операционных затрат и повышении удовлетворенности как клиентов, так и сотрудников службы поддержки.

Предлагаемый подход к решению проблемы базируется на концепции структурированной иерархической базы знаний в сочетании с механизмами интеллектуального поиска и последовательного уточнения запросов описанного в предыдущем разделе. Система спроектирована с учетом возможности горизонтального масштабирования и адаптации к различным предметным областям, что делает ее универсальной платформой для автоматизации процессов, требующих интеллектуального анализа структурированной информации и принятия решений на ее основе.

2.1 Концептуальная модель системы управления обращениями

Система управления обращениями предназначена для упрощения взаимодействия с пользователями, автоматизации процессов обработки обращений и повышения эффективности работы операторов. Основной принцип разработки — минимализм в интерфейсе при высокой интеллектуальной насыщенности функционала. Также система изначально проектируется таким образом, чтобы в дальнейшем имелась возможность внедрять её для решения задач в других предметных областях.

Концептуальная модель интеллектуальной системы управления потоками пользовательских обращений, рисунок 2.1, основывается на ряде фундаментальных принципов, определяющих её архитектуру и функциональные возможности:

1. В основе системы лежит принцип организации информации в виде n-арного дерева (графа), хранящегося в базе знаний. Каждый узел этого дерева представляет определенную категорию, подкатегорию или конкретное

решение/фрагмент знания. Такая структура обеспечивает логическую связность информации и позволяет реализовать механизм последовательного углубления в предметную область при поиске решения.

2. Система проектируется таким образом, чтобы каждый шаг в процессе поиска решения был прозрачен и объясним. Для каждого ответа фиксируется точный путь в иерархии знаний, по которому он был получен. Это обеспечивает полную прослеживаемость процесса принятия решений и позволяет пользователю понять логику системы.

3. LLM в системе выполняют роль интеллектуальных ассистентов-навигаторов по базе знаний, а не генераторов контента. Они интерпретируют естественно-языковые запросы пользователей, определяют релевантные пути поиска в дереве знаний и формируют диалог для уточнения контекста, но не создают ответы самостоятельно из неструктурированных данных. Это исключает проблему "галлюцинации" и неконтролируемых ответов, характерную для чисто генеративных моделей.

4. Система предоставляет администраторам и экспертам предметной области инструменты для самостоятельной настройки логики обработки запросов и структуры базы знаний без необходимости изменения программного кода. Это критически важно для быстрой адаптации к изменениям в бизнес-процессах или применительно к новым предметным областям. Базовые механизмы работы (поиск, навигация, формирование структурированного ответа) остаются неизменными при смене предметной области, что достигается за счет универсальной модели представления знаний.

5. Одним из ключевых требований является простота пользовательского интерфейса. За счёт интеллектуализации элементов управления минимизируется количество необходимых действий со стороны пользователя, обеспечивается интуитивное взаимодействие. Функциональность четко разделяется между интерфейсом для конечных пользователей (клиентским порталом) и интерфейсом для операторов и администраторов.

6. Архитектура системы изначально рассчитана на поддержку горизонтального и вертикального масштабирования, что позволит ей справляться с увеличивающейся нагрузкой. Это обеспечивается за счёт поддержки кластеризации, возможности работы с распределёнными базами данных и гибкости интеграции с внешними сервисами. Система предназначена для организации мультиканального взаимодействия с пользователями через платформы, такие как Telegram-бот, веб-приложение или электронная почта. Для каждой платформы разрабатываются адаптеры, преобразующие входящие запросы в унифицированный формат для обработки ядром системы.

Все процессы должны быть интуитивно понятны, автоматизированы и минимизированы с точки зрения сложности взаимодействия пользователя с системой. Архитектура включает несколько взаимосвязанных компонентов,

обеспечивающих полный цикл обработки пользовательских обращений.

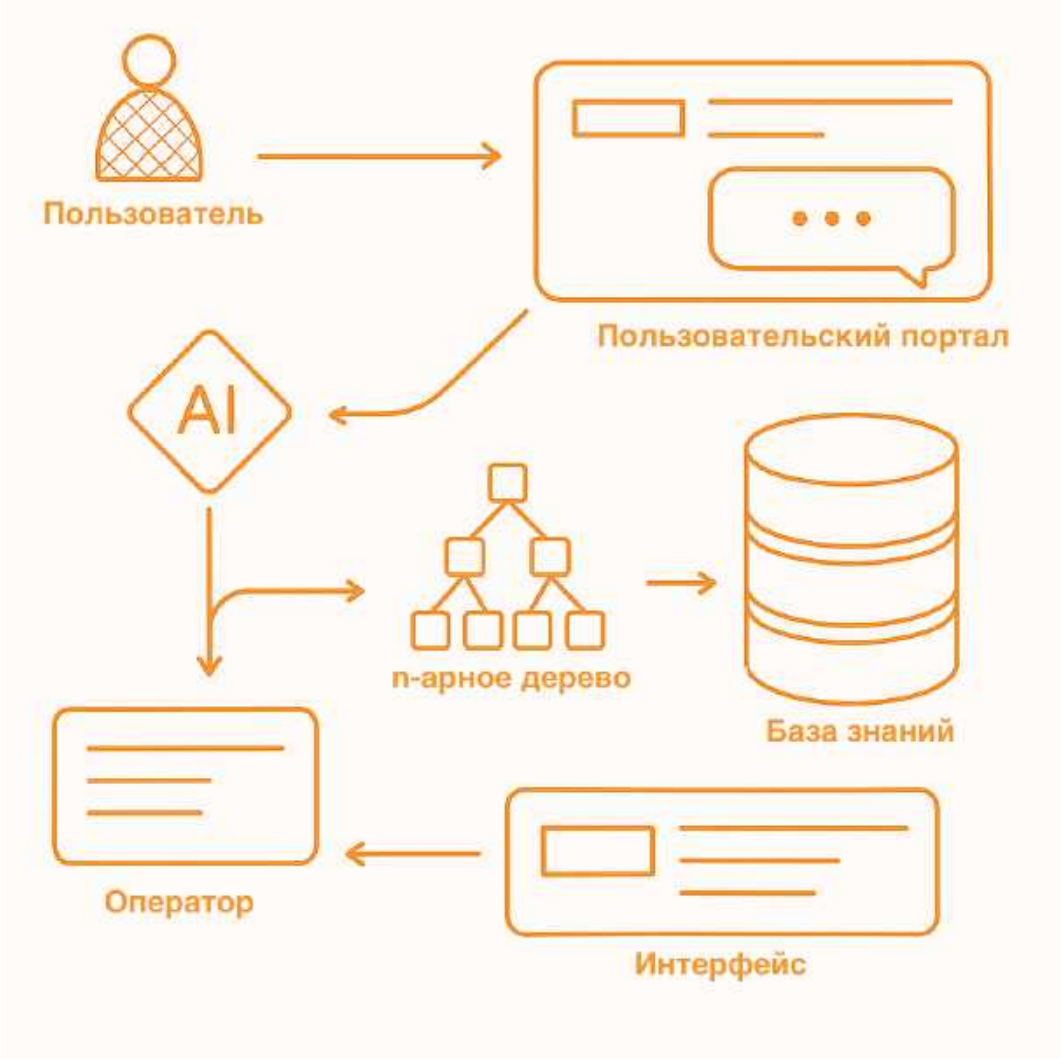


Рисунок 2.1 – Концептуальная модель системы управления обращениями

Проектируемая система предназначена для организации мультиканального взаимодействия с пользователями через такие платформы, как, например, Telegram-бот, веб-приложение или электронная почта. Для каждой из этих платформ планируется разработка отдельных адаптеров, которые будут преобразовывать входящие запросы в унифицированный формат и работать посредствам WS-API (Web Socket[14]). Оно будет служить основной точкой входа для всех пользовательских сообщений и обеспечивать маршрутизацию запросов к внутренним модулям обработки.

На уровне бизнес-логики предполагается разделение функций между двумя основными модулями: модулем обработки запросов и модулем выполнения команд. Модуль запросов будет отвечать за получение и предоставление пользователям необходимой информации, тогда как модуль команд будет предназначен для обработки управляющих действий. Оба модуля

будут взаимодействовать с брокером сообщений, который обеспечит асинхронную и масштабируемую обработку данных внутри системы.

Для поддержки интеллектуальной обработки запросов предусматривается создание модуля поиска решений. Этот модуль будет работать с базой знаний для получения постоянной информации и с системой хранения временной/контекстной информации для учета пользовательского состояния и истории взаимодействия. Также предполагается возможность интеграции с внешними сервисами для расширения функциональности и получения дополнительных данных.

Вся архитектура проектируется на основе событийно-ориентированного подхода, что обеспечит гибкость, масштабируемость и возможность последующего развития системы без необходимости значительных изменений в её базовых компонентах.

2.2 Варианты использования пользвателями системы

Интеллектуальная система управления потоками пользовательских обращений предоставляет оптимизированный интерфейс взаимодействия для трех основных ролей: конечных пользователей, операторов поддержки и администраторов базы знаний. Рассмотрим ключевые сценарии использования системы на примере UML диаграмм вариантов использования.

2.2.1 Основные сценарии работы с системой

Центральными участниками процесса обработки обращений являются пользователи и операторы поддержки, взаимодействие которых с системой представлено на рисунке 2.2 UML [15] диаграммы Usecase [16].

Взаимодействие пользователя с системой начинается с создания обращения через один из доступных каналов связи. Ключевой особенностью системы является возможность выбора шаблона обращения, что позволяет стандартизировать процесс подачи запросов и улучшить качество их последующей обработки. Шаблоны содержат предварительно структурированные поля для наиболее распространенных типов обращений, что упрощает формулировку запроса для пользователя и повышает точность автоматической обработки.

После создания обращения активируется интеллектуальный алгоритм поиска решения. Система анализирует содержание запроса, определяет его категорию и осуществляет навигацию по иерархической базе знаний с использованием языковых моделей в качестве интеллектуальных навигаторов. В результате пользователь получает автоматический ответ, который включает не только текст решения, но и визуализацию пути поиска в базе знаний, обеспечивая полную прозрачность процесса принятия решения.



Рисунок 2.2 – Диаграмма вариантов использования основной системы

Важным аспектом системы является возможность запроса дополнительной информации. Когда система определяет недостаток контекста для принятия точного решения, она автоматически формирует уточняющие вопросы, направляя пользователя к предоставлению необходимых деталей. Это позволяет избежать предоставления нерелевантной информации и повысить качество финального ответа.

Пользователь также имеет возможность отслеживать статус обращения на всех этапах его обработки и оценивать качество полученного ответа, что обеспечивает обратную связь для непрерывного улучшения системы.

Роль оператора задействуется в случаях, когда система не может автоматически разрешить обращение. Преимуществом для оператора является доступ к полной истории поиска решения, предпринятого системой. Это включает информацию о пройденных узлах базы знаний, выполненных запросах к языковым моделям и всех уточняющих вопросах. Такой подход значительно сокращает время, необходимое оператору для понимания сути проблемы и поиска эффективного решения.

2.2.2 Администрирование системы

Для поддержания актуальности и эффективности системы предусмотрена специальная роль администратора базы знаний, функциональность которого представлена на рисунке 2.3.



Рисунок 2.3 – Диаграмма вариантов использования администрирования системы

Управление базой знаний представляет собой центральную функциональность административного интерфейса. Администратор может осуществ-

лять просмотр и навигацию по иерархической структуре знаний, добавлять новые узлы информации, редактировать существующие записи и управлять связями между различными элементами базы знаний. Система также предоставляет возможность удаления устаревших или неактуальных узлов, обеспечивая поддержание базы знаний в актуальном состоянии.

Конфигурация поиска и ответов позволяет администратору тонко настраивать работу интеллектуальных механизмов системы. Это включает настройку параметров LLM-навигатора, корректировку шаблонов промптов для различных сценариев взаимодействия с языковыми моделями, а также установку порогов релевантности для автоматического принятия решений о качестве найденных ответов.

Мониторинг качества базы знаний представляет перспективную функциональность, которая позволит администратору анализировать частоту использования различных узлов базы знаний и оценивать эффективность различных путей решения проблем. Эта информация будет использоваться для оптимизации структуры знаний и улучшения качества автоматических ответов.

2.3 Проектирование модели данных

Для эффективного функционирования интеллектуальной системы, концептуальная модель которой была представлена ранее, необходима гибкая, расширяемая и абстрактная модель данных. Эта модель должна служить основой для структурированного хранения знаний, управления процессами взаимодействия и обеспечивать возможность адаптации ядра системы к различным прикладным задачам и предметным областям. Основу такой обобщенной модели данных составляют следующие ключевые сущности:

1. Информационный узел. Базовая атомарная единица структурированных данных в иерархической системе. Представляет собой элемент в графе знаний, обладающий уникальным идентификатором, наименованием, описательным контентом и типизированными связями с другими узлами. Может представлять концепцию, факт, правило, категорию или конкретный фрагмент данных. Атрибуты узла могут включать его текстовое описание, метаданные и ссылки на дочерние или связанные узлы.

2. Иерархическая связь. Определяет типизированные отношения между Информационными Узлами (например, отношение декомпозиции), формируя древовидную или более сложную графовую структуру. Эти связи обеспечивают логическую навигацию, декомпозицию сложных понятий и структурирование предметной области, что является основой для алгоритмов последовательного углубления.

3. Сессия зпроса. Представляет собой полный цикл обработки одного входящего запроса от его инициации до получения конечного результата.

Эта сущность инкапсулирует исходные данные запроса, текущее состояние процесса поиска (например, количество итераций, флаги завершения), историю навигации по Информационным Узлам (посещенные узлы, стек узлов для дальнейшего исследования), накопленный диалоговый контекст (история обмена сообщениями с LLM, комментарии LLM) и итоговый результат или указание на невозможность его нахождения.

4. Шаблон. Предопределенная конфигурация или начальные параметры для обработки часто встречающихся типов запросов или задач. Хотя это может быть реализовано на прикладном уровне, ядро системы должно поддерживать возможность инициализации сессии запроса с учетом таких сценариев для ускорения обработки.

5. Модель состояния сессии. Определяет различные фазы или состояния, через которые проходит Сессия Запроса в ядре системы. Логика переходов между состояниями управляется ядром на основе результатов взаимодействия с базой знаний и LLM.

6. Журнал операций сессии. Детализированная запись всех ключевых событий и действий, происходящих в рамках одной Сессии Запроса. Включает шаги навигации по графу знаний, запросы к LLM и их ответы, а также финальные комментарии LLM, объясняющие весь путь к решению. Обеспечивает отслеживаемость, возможность анализа и отладки работы ядра.

7. Журнал событий — лог взаимодействий пользователей, операторов и системы, фиксирующий все изменения в обращениях, отправленные ответы, смену статусов и другие важные действия. Журнал позволяет отслеживать историю обработки каждого запроса и обеспечивает прозрачность процессов.

Эти обобщенные сущности формируют универсальную основу модели данных для интеллектуального ядра системы, способного осуществлять навигационный поиск по структурированным знаниям.

При разработке конкретного приложения на базе этого ядра, такого как система управления потоками пользовательских обращений, описанные выше абстрактные сущности находят следующую имплементацию:

1. Информационный узел конкретизируется как статья базы поддержки, описание типичной проблемы, шаг в инструкции по устранению неполадки, информация о продукте или услуге.

2. Иерархическая связь определяет логические переходы между статьями, детализацию проблем или последовательность шагов решения.

3. Сессия запроса становится «Обращением» или «Заявкой пользователя», инициированной через один из каналов (например, Telegram-бот).

4. Типовой Сценарий преобразуется в «Шаблон обращения» (например, для запроса “сброс пароля“ или “узнать статус заказа“), который может предопределять начальные узлы для поиска или специфические промпты

для LLM.

5. Модель состояния Ссссии дополняется специфичными для управления обращениями статусами.

6. Журнал Операций Сессии становится частью истории обработки конкретного обращения, доступной как пользователю (в части пути решения), так и оператору (полная история для анализа).

Такой подход, разделяющий абстрактное ядро и его прикладное использование, обеспечивает высокую степень переиспользуемости разработанного движка и его адаптивность к различным задачам, требующим интеллектуальной навигации по структурированным базам знаний.

2.4 Интеллектуальные механизмы обработки обращений

Предлагаемая система автоматизирует поддержку пользователей, опираясь на иерархическую базу знаний и обеспечивая полную прозрачность процесса принятия решений. В отличие от “черных ящиков“ традиционного AI, каждый этап формирования ответа здесь отслеживаем.

Ключевой элемент системы – модуль поиска решений. Он использует алгоритм последовательного углубления для навигации по верифицированным данным в иерархической базе знаний, а не генерирует ответы самостоятельно. Такой подход исключает “галлюцинации“, свойственные генеративным языковым моделям (см. рисунок 2.4).

Процесс обработки обращения включает несколько ключевых этапов:

1. Первичная классификация запроса - на этом этапе система определяет общую категорию обращения и выбирает начальную точку в иерархической структуре знаний. Классификация осуществляется с использованием языковых моделей, которые интерпретируют естественно-языковой запрос и сопоставляют его с существующими категориями.

2. Последовательное уточнение и навигация - система перемещается по дереву знаний, на каждом шаге определяя наиболее релевантные ветви для дальнейшего исследования. При необходимости формируются уточняющие вопросы пользователю для сужения области поиска. Важно отметить, что весь путь навигации фиксируется и может быть представлен как обоснование финального решения.

3. Формирование структурированного ответа - при нахождении релевантной информации система формирует ответ на основе данных из базы знаний с обязательным указанием пути в иерархии, по которому был получен этот ответ. Такой подход обеспечивает прозрачность и позволяет пользователю понять, почему система предложила именно это решение.

4. Анализ необходимости эскалации - если система не может найти решение в базе знаний или определяет, что проблема требует вмешательства специалиста, активируется механизм интеллектуальной эскалации. При

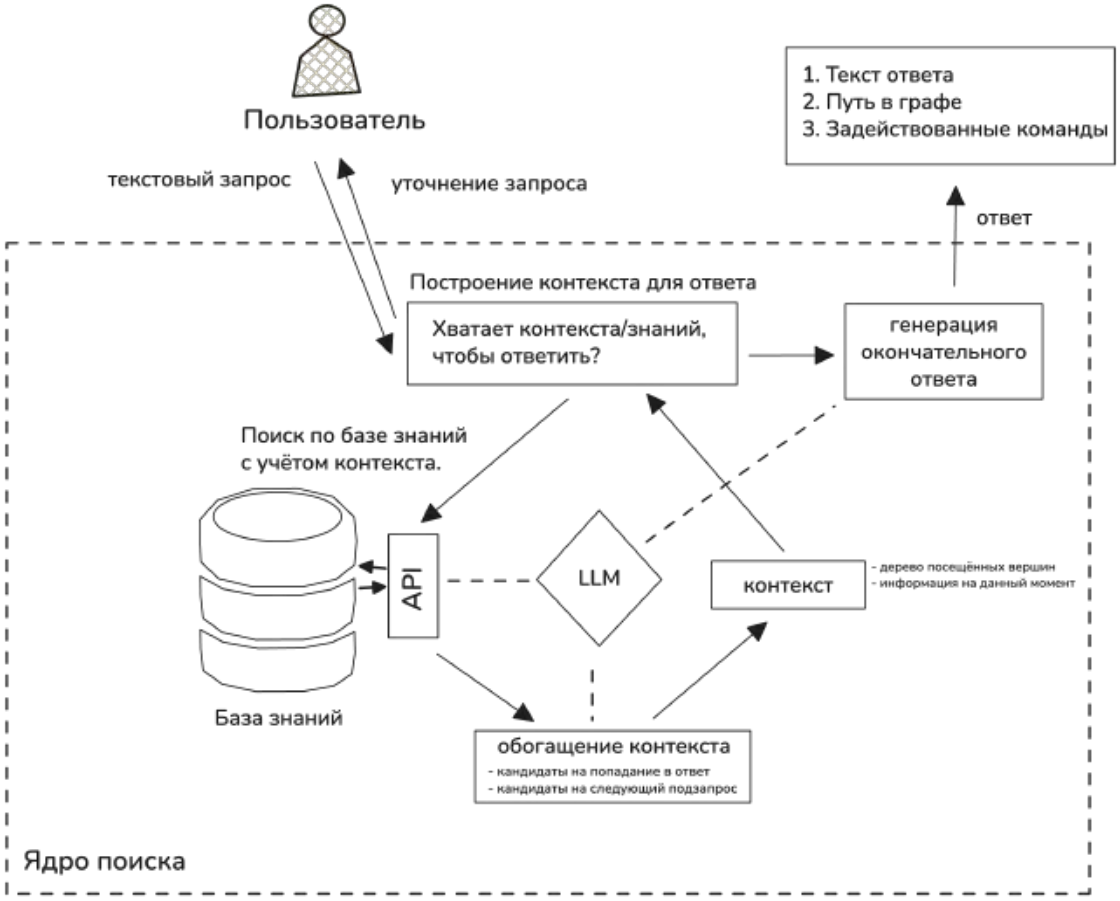


Рисунок 2.4 – Схема работы алгоритма интеллектуального поиска

этом вся собранная в процессе диалога информация и пройденный путь поиска передаются оператору, что существенно сокращает время обработки обращения.

Важно отметить, что интеллектуальные механизмы системы разработаны с учетом возможности их настройки без программирования. Администраторы могут самостоятельно конфигурировать правила классификации, структуру базы знаний и логику эскалации через интуитивно понятный интерфейс, что обеспечивает гибкость и адаптивность решения к меняющимся бизнес-требованиям.

Визуально структура работы алгоритма интеллектуального поиска представлена на рисунке 2.5. Диаграмма иллюстрирует поэтапное движение запроса пользователя по иерархической базе знаний: от начальной классификации и выбора релевантной ветви до последовательного уточнения и формирования финального ответа. Каждый этап сопровождается фиксацией принятых решений и, при необходимости, формированием дополнительных запросов для уточнения информации. Такой подход обеспечивает не только прозрачность логики поиска, но и возможность детального аудита всех действий системы.

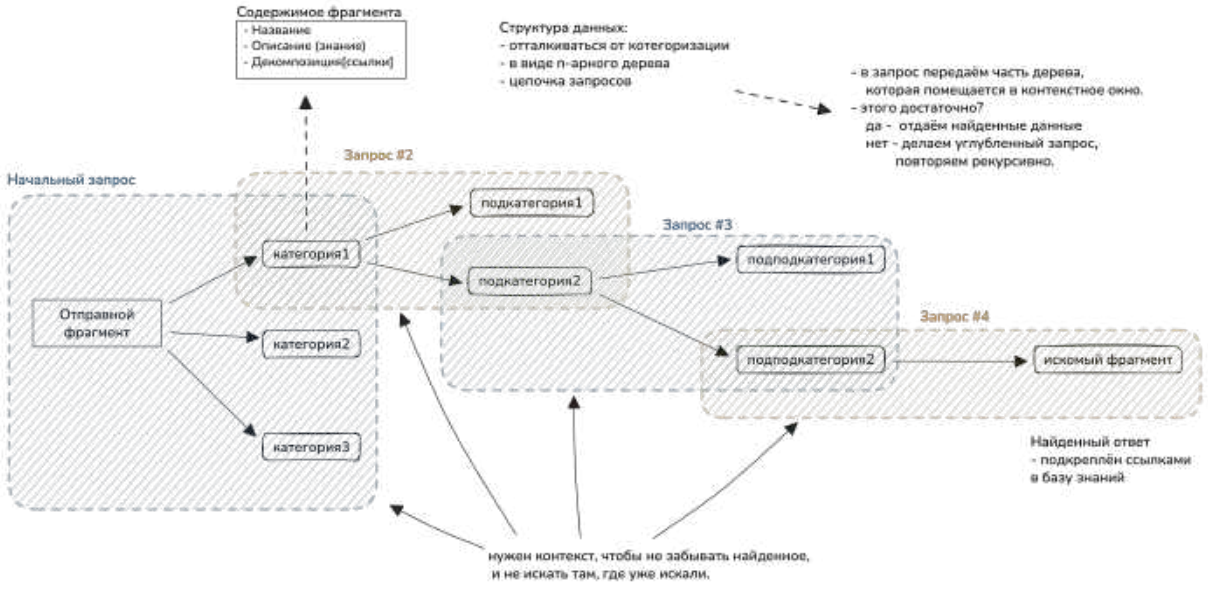


Рисунок 2.5 – Пример работы интеллектуального поиска

Использование подобной схемы позволяет наглядно продемонстрировать, каким образом система избегает типичных ошибок генеративных моделей, опираясь исключительно на проверенные знания и четко структурированные правила навигации. Благодаря этому пользователь всегда может проследить, каким образом был получен ответ, а администратор — оперативно выявить и скорректировать возможные узкие места в логике обработки обращений.

Для дальнейшей детализации этого алгоритма интеллектуального поиска, рассмотрим его работу с двух дополнительных ракурсов. Рисунок 2.6 иллюстрирует пошаговую логику процесса, акцентируя внимание на итеративном характере навигации по базе знаний. На каждом шаге LLM анализирует текущий контекст и доступные узлы, рекомендуя либо углубление в иерархии, либо предоставление финального ответа, либо указывая на нехватку информации. Важным аспектом является фиксация всего пути решения для обеспечения прозрачности.

В свою очередь, рисунок 2.7 демонстрирует временное взаимодействие между ключевыми компонентами: пользователем (через адаптер, например, Telegram-бот), ядром системы, LLM и базой знаний (Neo4j). Эта диаграмма наглядно прослеживает весь путь запроса от его инициации пользователем, через обработку ядром и итеративные консультации с LLM, до формирования и доставки финального структурированного ответа.

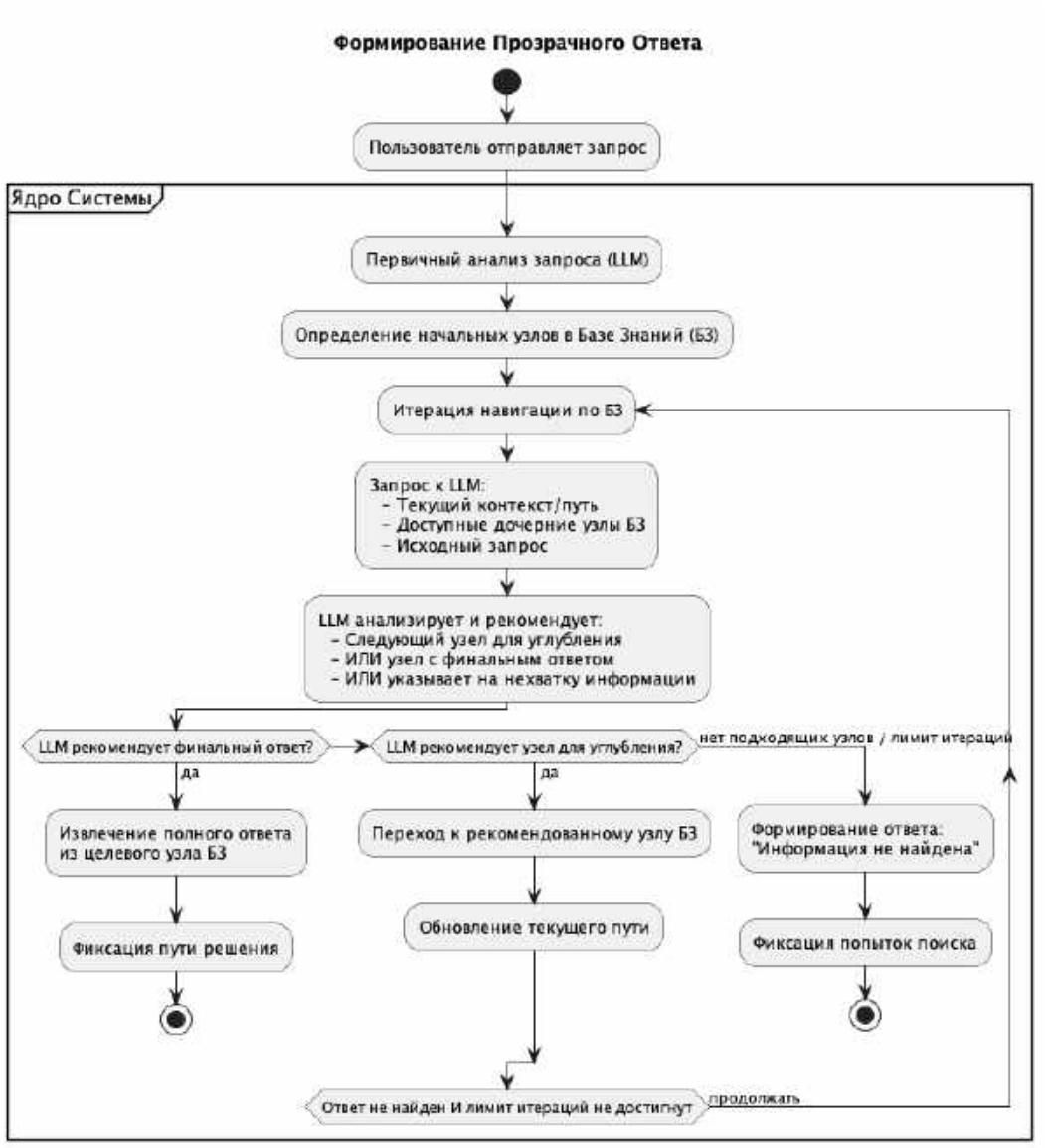


Рисунок 2.6 – Диаграмма последовательности последовательность обработки запроса

2.4.1 Автоматическая обработка обращений

Когда пользователь создаёт новое обращение через клиентский портал, система передаёт его данные на обработку, рисунок 2.8. Первым этапом задействуется модуль ИИ, который анализирует текст запроса, определяет его основную суть и пытается сопоставить с уже существующими обращениями и шаблонами решений.

Если система находит подходящий ответ, она предлагает его пользователю в виде автоматического решения. Это может быть текстовое пояснение, ссылка на статью из базы знаний или пошаговая инструкция. Пользователь получает возможность либо принять предложенное решение, либо отклонить его, если оно не соответствует его ожиданиям.

Если ответ оказался полезным, пользователь подтверждает его при-

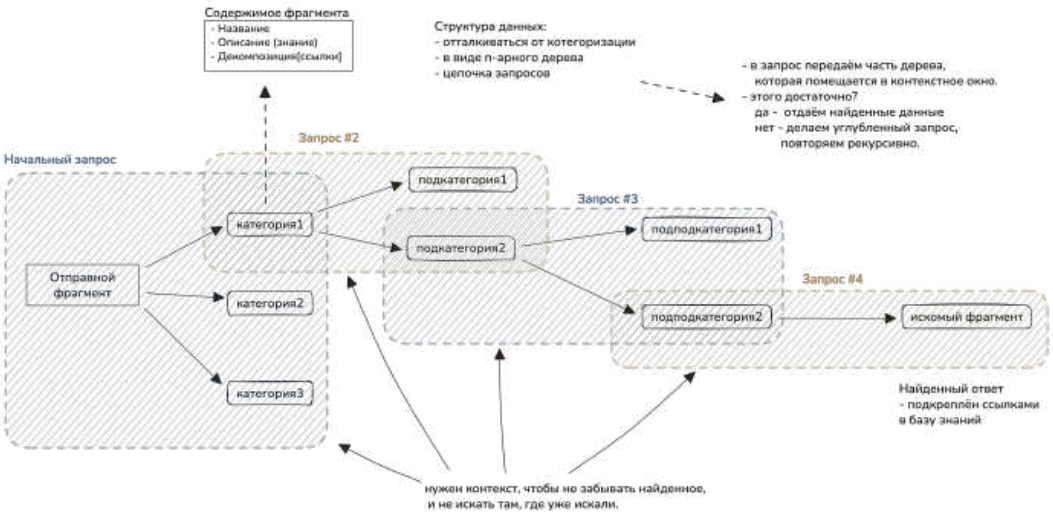


Рисунок 2.7 – Диаграмма последовательности последовательность обработки запроса

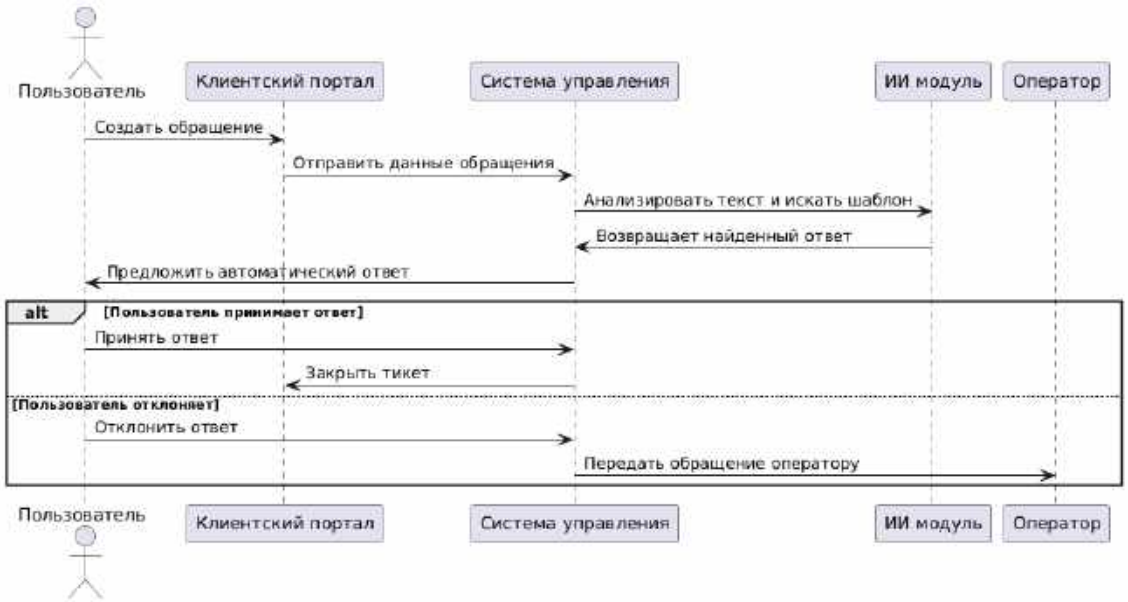


Рисунок 2.8 – Диаграмма последовательности автозакрытия обращения

нятие, и система автоматически закрывает запрос, избавляя оператора от необходимости вручную обрабатывать обращение. В случае отказа система передаёт запрос оператору, который уже вручную анализирует проблему и взаимодействует с пользователем.

Этот механизм обеспечивает баланс между автоматизацией и персонализированным подходом, позволяя в простых случаях мгновенно решать запросы, а в сложных — передавать их специалистам. Использование ИИ не только ускоряет процесс обработки обращений, но и позволяет системе постоянно обучаться, улучшая точность предлагаемых решений на основе накопленного опыта.

2.4.2 Прогнозирование эскалаций

Одной из ключевых интеллектуальных функций системы является механизм прогнозирования эскалаций и автоматической подготовки ответов для операторов, рисунок 2.9. Используя искусственный интеллект, система анализирует содержание обращений, выявляет потенциально сложные или критические запросы и предсказывает вероятность их эскалации. Это позволяет снизить нагрузку на поддержку и повысить уровень удовлетворённости пользователей за счёт быстрого и точного реагирования.



Рисунок 2.9 – Диаграмма последовательности прогнозирования эскалаций

При создании нового обращения система оценивает его сложность, анализируя тональность сообщения, историю взаимодействий пользователя, частоту прошлых эскалаций и текущую загруженность операторов. Если система обнаруживает высокий риск эскалации, она выполняет два ключевых действия. Во-первых, оператору автоматически предлагаются возможные решения на основе анализа предыдущих аналогичных запросов, что позволяет ему быстрее сформировать ответ. Во-вторых, если вероятность эскалации особенно высока, система повышает приоритет обращения, уведомляет старших специалистов или предлагает пользователю альтернативные способы решения проблемы, такие как запись на звонок с экспертом. Благодаря этому механизму обработка обращений становится более предсказуемой, а операторы могут эффективнее распределять свое время, снижая количество нерешённых запросов и улучшая качество обслуживания.

2.5 Архитектурные решения системы

Архитектура системы управления обращениями разработана с акцентом на асинхронное взаимодействие, масштабируемость и изоляцию

ключевых компонентов. Основной принцип проектирования — обеспечение высокой отказоустойчивости и гибкости за счёт использования брокера сообщений, разделения потоков команд и запросов, а также минимизации прямых связей между сервисами, рисунок 2.10. Подробнее этот и другие паттерны описаны в книге “Шаблоны интеграции корпоративных приложений“[17].

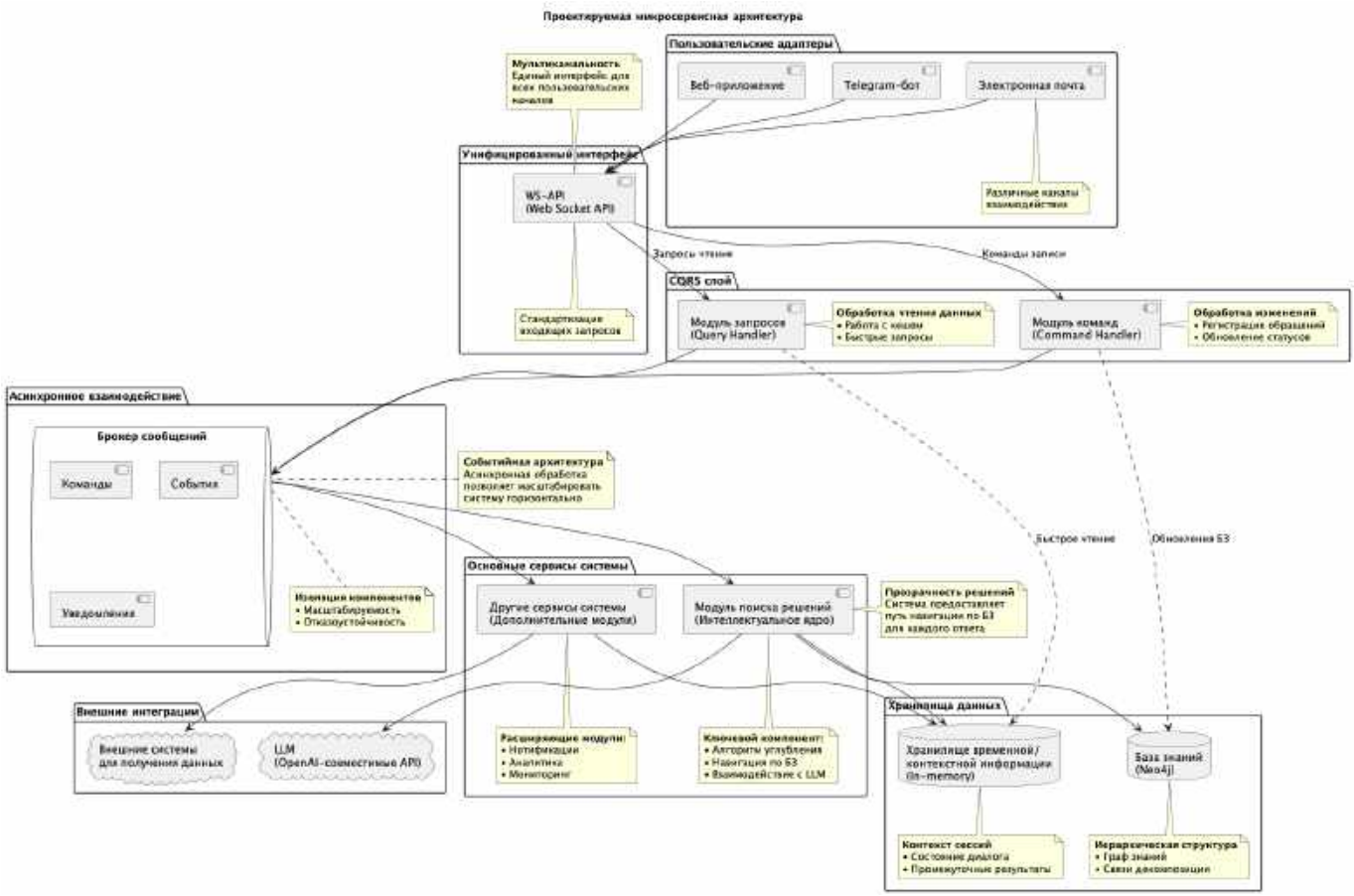


Рисунок 2.10 – Архитектура системы

Архитектура включает следующие взаимосвязанные компоненты, обеспечивающие полный цикл обработки пользовательских обращений:

1. Пользовательские адаптеры - обеспечивают взаимодействие с пользователями через различные каналы коммуникации.

2. WS-API - унифицированный интерфейс для взаимодействия между пользовательскими адаптерами и внутренними модулями системы, обеспечивающий стандартизацию входящих запросов.

3. Модуль запросов - компонент, отвечающий за обработку входящих запросов от пользователей, их первичную классификацию и маршрутизацию.

4. Модуль команд - реализует логику выполнения конкретных действий в системе, таких как регистрация новых обращений, обновление

статусов и т.д.

5. Брокер сообщений - обеспечивает асинхронное взаимодействие между компонентами системы, повышая её отказоустойчивость и масштабируемость.

6. Модуль поиска решения - ключевой интеллектуальный компонент системы, реализующий алгоритм последовательного углубления по структуре базы знаний и формирования ответов.

7. База знаний - хранилище структурированной информации о типовых проблемах и их решениях в виде иерархической структуры.

8. Хранилище временной/контекстной информации - компонент для сохранения состояния диалога с пользователем и промежуточных результатов поиска, реализованный на базе высокопроизводительного in-memory хранилища.

9. Другие сервисы системы - дополнительные модули, расширяющие функциональность основной системы, которые также взаимодействуют через брокер сообщений.

Взаимодействие компонентов организовано в соответствии с принципами событийно-ориентированной архитектуры (подробнее про событийно-ориентированную архитектуру[18]) с использованием брокера сообщений в качестве центрального механизма коммуникации. Жизненный цикл обращения начинается с его поступления через один из пользовательских адаптеров, после чего запрос преобразуется в стандартизированный формат через WS-API и передается в модуль запросов. Далее модуль запросов совместно с модулем поиска решения осуществляют навигацию по базе знаний, при необходимости запрашивая дополнительную информацию у пользователя. Контекст диалога и промежуточные результаты сохраняются в хранилище временной информации. При нахождении решения формируется соответствующий ответ, который передается обратно пользователю через исходный канал коммуникации.

Разделение системы на модуль запросов и модуль команд соответствует принципу CQRS (Command Query Responsibility Segregation), что обеспечивает более эффективную масштабируемость компонентов в соответствии с характером нагрузки.

2.5.1 Разделение команд и запросов (CQRS)

В системе использован паттерн Command and Query Responsibility Segregation (CQRS [19]), который разделяет обработку команд (изменяющих состояние системы) и запросов (читающих данные). Такой подход позволяет оптимизировать производительность, упрощает масштабирование отдельных частей системы и даёт возможность использовать разные модели хранения данных для чтения и записи.

Команды отправляются через брокер сообщений и передаются в основной сервис управления обращениями, который выполняет их обработку, вносит изменения в систему и отправляет соответствующие события другим сервисам. Запросы же обрабатываются непосредственно через специализированные модули, работающие с кешем и постоянным хранилищем данных.

2.5.2 Изоляция главного сервиса через брокер сообщений

Центральным элементом системы является основной сервис управления обращениями, который отвечает за обработку обращений, управление их жизненным циклом и взаимодействие с другими компонентами. В целях обеспечения гибкости и отказоустойчивости он изолирован от остальных сервисов с помощью брокера сообщений. Все команды и события передаются через неё, позволяя системе масштабироваться горизонтально и предотвращая блокировки в случае увеличения нагрузки, рисунок 2.11.

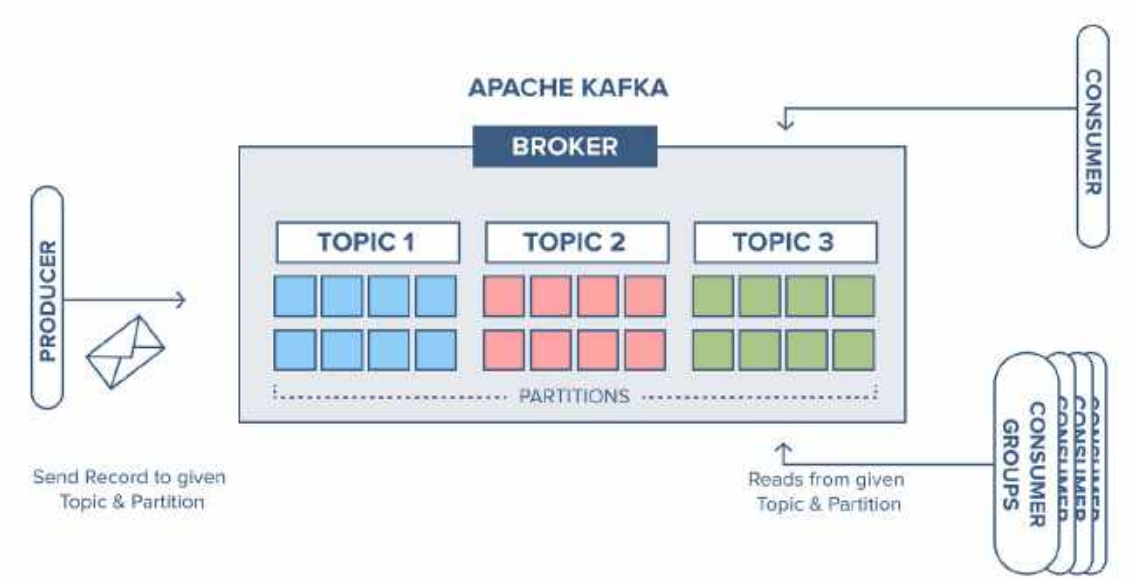


Рисунок 2.11 – Брокер сообщений

2.5.3 Дополнительные архитектурные компоненты системы

Помимо основного сервиса, система включает:

– модуль запросов, обрабатывающий чтение данных с использованием распределённого кеша для ускорения работы;

– модуль команд, отвечающий за обработку измений в системе;

– модуль нотификаций, отправляющий уведомления пользователям и осуществляющий обратную связь;

– логирование и метрики, регистрирующие всю активность системы для мониторинга и диагностики.

Для обеспечения мониторинга и отладки системы управления обращениями испльзуется подсистема логирования и сбора метрик. Логирование охватывает все ключевые события, включая создание и обработку обращений, работу интеллектуальных механизмов и взаимодействие с внешними сервисами. Логи записываются в централизованное хранилище и доступны для анализа в реальном времени.

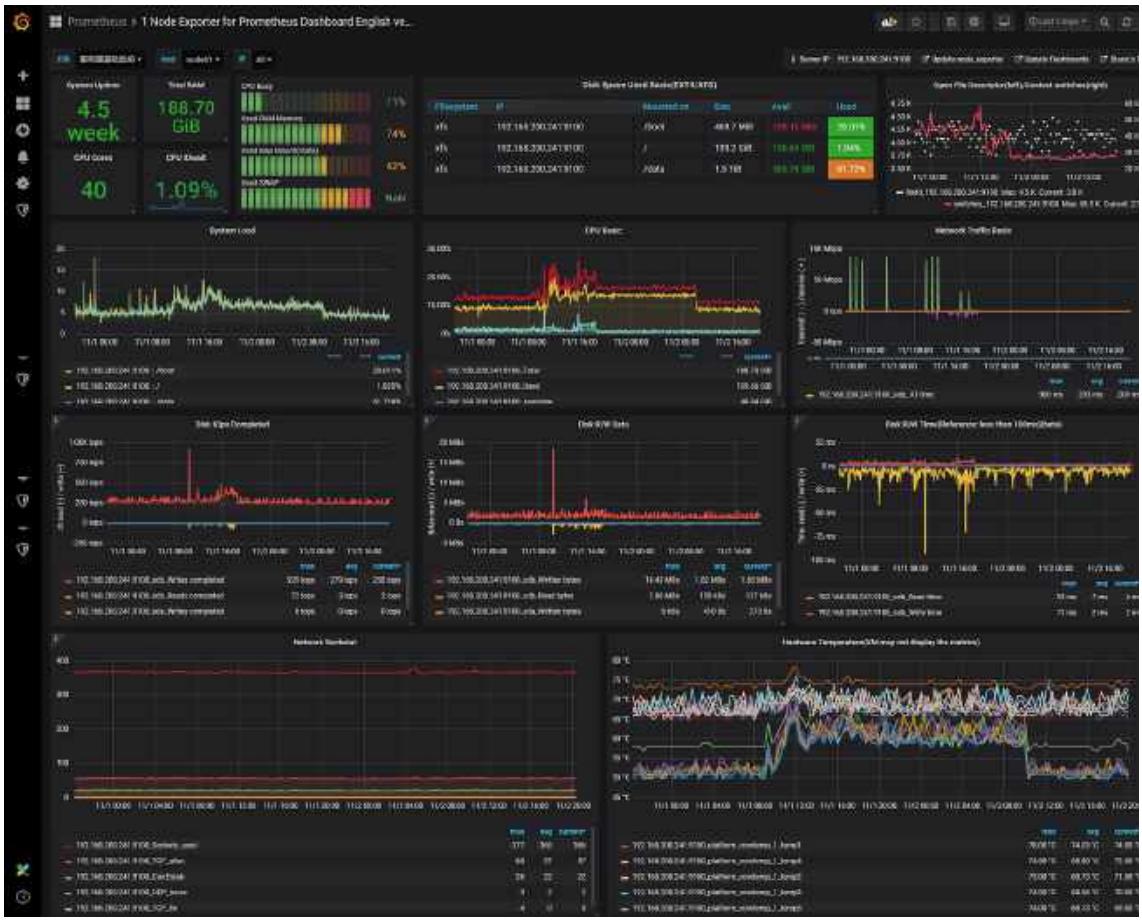


Рисунок 2.12 – Мониторинг метрик в системе

Дополнительно была интегрирована система сбора метрик на основе Prometheus, рисунок 2.12, позволяющая отслеживать производительность системы, нагрузку на сервер, время обработки запросов и частоту эскалаций обращений. Метрики агрегируются и визуализируются с помощью Grafana [20], что позволяет оперативно реагировать на потенциальные проблемы и оптимизировать работу системы. Такой подход обеспечивает прозрачность работы сервиса, а также упрощает масштабирование и поиск узких мест в архитектуре.

Также предусмотрено взаимодействие с ML-модулем, который интегрирован с облачными сервисами и отвечает за интеллектуальную обработ-

ку обращений. Это позволяет автоматизировать классификацию запросов, предлагать готовые ответы и предсказывать возможные эскалации.

Такой архитектурный подход делает систему устойчивой к высокой нагрузке, масштабируемой и удобной для интеграции с внешними сервисами. В ходе проектирования системы управления обращениями было принято решение использовать подходы разделения модулей на слои с чётким разграничением ответственности, что обеспечило высокую гибкость и независимость различных компонентов. В частности, была реализована архитектурная модель, в которой бизнес-логика системы была отделена от инфраструктурных деталей, таких как взаимодействие с базой данных и внешними сервисами [21].

2.6 Вывод

Проектирование интеллектуальной системы управления потоками пользовательских обращений требует сочетания гибкости, автоматизации и высокой производительности. В рамках данной архитектуры были применены современные подходы, обеспечивающие не только удобное взаимодействие с пользователем, но и высокую эффективность обработки обращений.

Принципиальным отличием разработанного решения является комплексный системный подход, который выходит за рамки традиционных представлений о сервисах технической поддержки. Современные технологические тренды диктуют необходимость создания интеллектуальных систем, способных не просто реагировать на запросы, но и предупреждать потенциальные проблемы, адаптироваться под изменяющиеся бизнес-процессы.

Ключевым аспектом системы является глубокая интеллектуализация процессов, включающая автоматический анализ обращений, предсказание решений с использованием машинного обучения и гибкую маршрутизацию запросов. Интеграция ИИ позволяет снизить нагрузку на операторов и ускорить решение типовых вопросов.

Для обеспечения высокой масштабируемости и отказоустойчивости в системе реализовано асинхронное взаимодействие через брокер сообщений Kafka. Применение паттерна разделения команд и запросов (CQRS) дало возможность оптимизировать производительность, а изоляция основного сервиса управления обращениями повысила надёжность всей системы.

Разработанная архитектура ориентирована на минимализм интерфейса, адаптивность и максимальную автоматизацию процессов. В результате система позволяет пользователям эффективно взаимодействовать с сервисом, а операторам — оперативно обрабатывать обращения, при этом оставаясь гибкой и расширяемой.

3 РЕАЛИЗАЦИЯ ИНТЕГРИРОВАННОЙ СИСИТЕМЫ УПРАВЛЕНИЯ ПОТОКАМИ ПОЛЬЗОВАТЕЛЬСКИХ ОБРАЩЕНИЙ

Данная глава посвящена описанию процесса разработки, начиная от выбора технологического стека и заканчивая имплементацией основных модулей и механизмов, которые воплощают в жизнь спроектированную концепцию.

В ходе изложения будут рассмотрены технические аспекты создания универсального ядра системы, способного осуществлять интеллектуальный навигационный поиск по иерархической базе знаний. Особое внимание будет уделено реализации алгоритма последовательного углубления, интеграции с языковыми моделями (LLM) в роли ассистентов-навигаторов, а также механизмам формирования объяснимых и структурированных ответов. Кроме того, будет продемонстрировано, как это ядро интегрируется с пользовательскими интерфейсами, на примере Telegram-бота и web-приложения, и как обеспечивается управление знаниями и контекстом взаимодействия. Цель данной главы – представить конечный программный продукт и раскрыть ключевые инженерные решения, принятые в процессе его создания.

3.1 Общие сведения о реализации

Практическая реализация интеллектуальной системы управления потоками пользовательских обращений потребовала выбора современного и гибкого технологического стека, а также четкой организации структуры проекта для обеспечения модульности и дальнейшего развития.

Для разработки ядра системы и сопутствующих компонентов был выбран язык TypeScript[22] в связке со средой выполнения Deno. Такое решение обусловлено преимуществами строгой типизации TypeScript, что критически важно для создания надежных и сложных систем, а также современными возможностями Deno, включая встроенную поддержку TypeScript, безопасность и удобное управление модулями.

Центральным элементом хранения знаний выступает графовая база данных Neo4j. Ее выбор продиктован способностью представлять иерархические данные в удобном для работы с ними формате, и идеально соответствует концепции дерева знаний системы. Взаимодействие с Neo4j из кода приложения реализовано посредством официального JavaScript-драйвера.

Интеллектуальная составляющая, отвечающая за навигацию по базе знаний и анализ запросов, опирается на большие языковые модели (LLM), доступ к которым осуществляется через API OpenAI-совместимых сервисов[23], в частности, с использованием модели deepseek-chat[24]. Для

интеграции с LLM используется соответствующая JavaScript/TypeScript библиотека.

В качестве демонстрационного пользовательского интерфейса и канала взаимодействия с системой был разработан Telegram-бот и простое web-приложение. Реализация Telegram-бота выполнена с помощью популярной библиотеки [25], предоставляющей удобные инструменты для создания ботов. Управление зависимостями проекта и его сборка осуществляются встроенными средствами Deno. Для обеспечения портативности и стандартизации окружения базы данных Neo4j используется Docker.

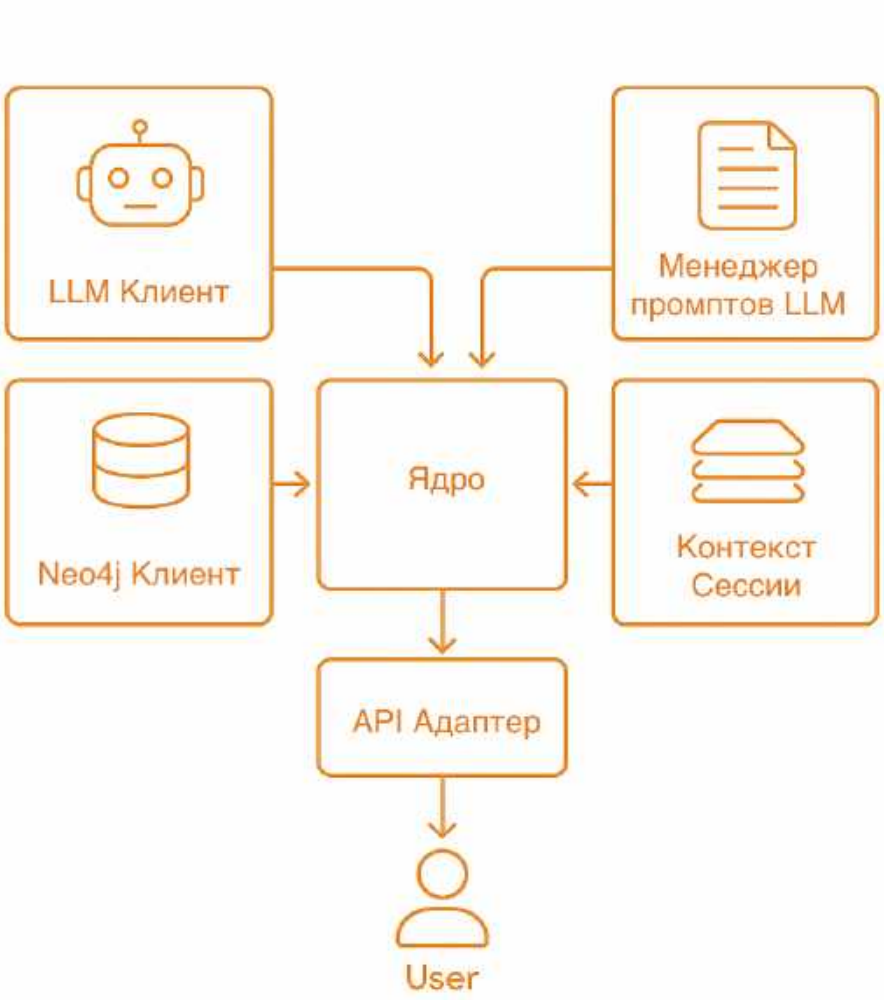


Рисунок 3.1 – Схема реализованного ядра системы

Также для взаимодействия с ядром системы обработки обращений было разработано веб-приложение с использованием React, TypeScript и Tailwind CSS[26]. Этот интерфейс предоставляет пользователям возможность отправлять запросы, отслеживать их статус обработки, просматривать историю обращений и визуализированный путь навигации системы по базе

знаний. Административная часть веб-приложения включает инструменты для управления базой знаний, включая визуализацию графа знаний и настройку параметров системы. Компоненты пользовательского интерфейса, такие как формы ввода, карточки отображения информации и элементы навигации, реализованы с использованием библиотеки shadcn/ui[27], обеспечивая современный и адаптивный дизайн. Управление состоянием приложения, включая историю запросов и текущую тему оформления, осуществляется через React Context API.

Структура исходного кода проекта была продумана для обеспечения модульности и четкого разделения ответственности. Центральным компонентом является engine, который инкапсулирует все ядро, включая логику взаимодействия с LLM, графовой базой данных Neo4j, управление промптами и основной класс Context, управляющий сессией поиска. Внешниq интерфейс, реализован на примере адаптера для Telegram-бота, обеспечивает взаимодействие пользователя с ядром, рисунок 3.1.

3.2 Реализация архитектуры системы

При практической разработке интеллектуальной системы управления потоками пользовательских обращений была реализована архитектура, представляющая собой минимально жизнеспособный продукт (MVP). Эта архитектура, представленная на 3.2, является функциональным ядром, основанным на принципах, изложенных в разделе 2, но сфокусированным на ключевых компонентах, необходимых для демонстрации основной концепции.

На текущем этапе реализации (MVP) система состоит из центрального интеллектуального ядра и двух пользовательских адаптеров (Telegram-бот и веб-приложение). Ядро инкапсулирует всю основную логику обработки запросов, взаимодействия с базой знаний (Neo4j) и языковыми моделями (LLM). Такая модульная архитектура, даже в рамках текущей монолитной реализации ядра, позволяет четко разделить зоны ответственности и упрощает дальнейшее развитие.

В перспективе, при росте нагрузок и функциональных требований, отдельные компоненты ядра или система целиком могут быть трансформированы в микросервисную архитектуру, где каждый сервис будет отвечать за свою узкоспециализированную задачу (например, сервис управления базой знаний, сервис оркестрации LLM-взаимодействий, сервис обработки пользовательских сессий). Подходы к постраению микросервисных архитектур можно рассмотреть в книге “Создание микросервисов“[28]. Однако для MVP была выбрана более простая и быстрая в реализации структура, позволяющая проверить основные гипотезы и продемонстрировать работоспособность ключевых механизмов.

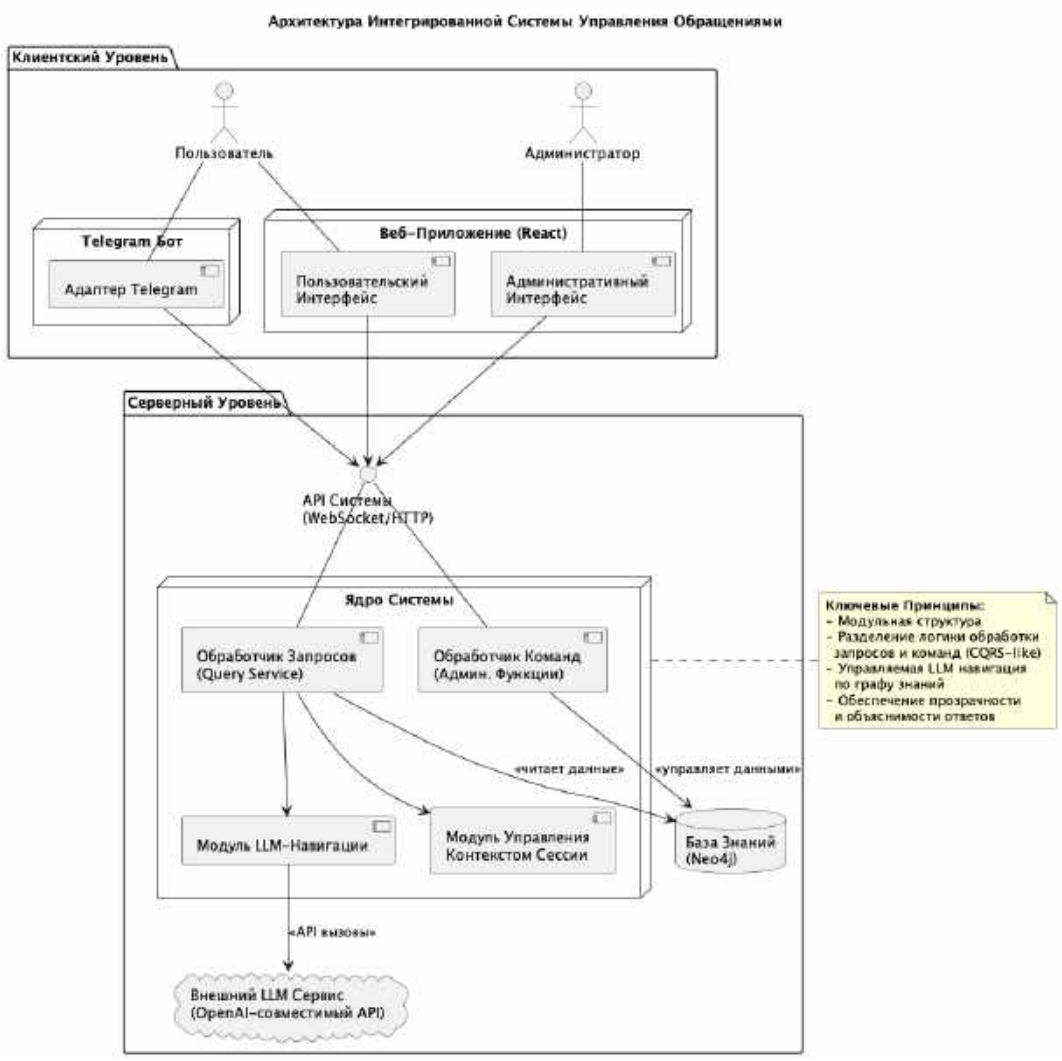


Рисунок 3.2 – Архитектура реализованного MVP интегрированной системы управления обращениями

3.2.1 Структура реализованных модулей и перспективы микросервисной архитектуры

В текущей реализации MVP ключевым программным модулем является интеллектуальное ядро. Этот модуль является сердцем системы и отвечает за управление сессией обработки запроса. Ядро также включает компоненты для взаимодействия с графовой базой данных Neo4j, необходимые для извлечения фрагментов знаний и модули для формирования запросов (промптов) к языковой модели и обработки ее ответов с целью навигации по базе знаний. Кроме того, ядро ведет историю взаимодействия и контекста в рамках одной сессии. Вторым важным модулем является адаптер Telegram-бота, который служит точкой входа для пользователей. Он принимает сообщения, инициализирует сессию обработки запроса ядра и передает итоговый ответ от ядра обратно пользователю.

В более развитой, микросервисной архитектуре, концепция которой была рассмотрена на этапе проектирования, эти функции были бы распределены между несколькими независимыми сервисами. Например,“Модуль запросов“ и “Модуль команд“ могли бы функционировать как отдельные микросервисы, взаимодействующие с ядром, которое, в свою очередь, также могло бы быть разделено на сервис поиска решений и сервис управления знаниями.“Пользовательские адаптеры“ для различных каналов, таких как Telegram, веб-интерфейсы или электронная почта, также представляли бы собой отдельные, независимо развертываемые сервисы.

3.2.2 Организация асинхронного взаимодействия

В текущей реализации ядра асинхронность является фундаментальным аспектом. Важно подчеркнуть, что JavaScript по своей природе является однопоточным и использует событийную модель (event loop) для управления операциями. Это означает, что в основном потоке выполнения нет истинной многопоточности, как в языках типа Java или C++. Вместо этого, JavaScript создает иллюзию параллелизма, которую часто называют “псевдо-многопоточностью“. Асинхронность в такой среде достигается за счет активного использования конструкций “async/await“. Эти конструкции позволяют регистрировать операции ввода-вывода (I/O), такие как взаимодействие с внешними системами (база данных Neo4j, API языковой модели), и передавать управление обратно событийному циклу, не блокируя основной поток. Таким образом, обеспечивается конкурентное выполнение задач, позволяя приложению оставаться отзывчивым, но не параллельное выполнение кода на нескольких ядрах процессора в рамках одного процесса.

Для полномасштабной, распределенной системы, как было описано в разделе ранее, организация асинхронного взаимодействия между микросервисами потребует дополнительного внедрения брокера сообщений, например, Apache Kafka[29]. Такое решение обеспечит надежную доставку сообщений между сервисами, их меньшую связность (когда сервисы-продюсеры не зависят напрямую от сервисов-консьюмеров), а также возможности для масштабирования за счет параллельной обработки сообщений (уже на уровне нескольких процессов или инстансов сервисов, где каждый инстанс может быть отдельным JS-процессом) и повышения отказоустойчивости. В подобной архитектуре могли бы использоваться специализированные топики Kafka для различных типов сообщений, таких как входящие запросы пользователей, задачи для LLM, события обновления базы знаний, логирование событий изменняия сессий и передача финальных ответов.

Хотя в текущем MVP Kafka не используется, архитектура ядра изначально проектировалась с учетом возможности последующего внедрения систем обмена сообщениями, таких как Kafka, что станет актуальным при

переходе к микросервисной архитектуре.

3.2.3 Реализация универсального API взаимодействия

Для обеспечения легкой интеграции интеллектуального ядра с разнообразными пользовательскими интерфейсами был реализован унифицированный программный API. Центральным элементом этого API является фабричный метод, который инициализирует сессию обработки запроса, возвращая объект контекст. Через публичные методы этого объекта любой внешний адаптер, будь то Telegram-бот, веб-приложение или другой сервис, может единообразно управлять процессом поиска и получать структурированные результаты. Такой подход значительно упрощает создание новых адаптеров и обеспечивает гибкость системы, закладывая основу для возможного перехода к более формальному сетевому API (например, REST или gRPC) в случае выделения ядра в отдельный микросервис в будущем.

3.3 Реализация подсистемы управления знаниями

Функционирование интеллектуальной системы неразрывно связано с эффективным управлением базой знаний (БЗ) и контекстом взаимодействия в процессе поиска. Качество и структура БЗ напрямую определяют релевантность и точность ответов, предоставляемых системой. Данный подраздел описывает ключевые аспекты реализации хранилища знаний и механизмов управления контекстной информацией.

3.3.1 Модель хранения иерархической базы знаний

Центральным элементом подсистемы управления знаниями является иерархическая база данных, для реализации которой была выбрана графовая СУБД Neo4j. Такой выбор обусловлен нативной поддержкой Neo4j графовых моделей данных, что идеально соответствует концепции представления знаний в виде дерева или более сложного графа. Язык запросов Cypher[30], используемый в Neo4j, оптимизирован для посика по связям между узлами, что является важным для реализации алгоритма последовательного углубления, применяемого в ядре системы. Гибкость схемы Neo4j также позволяет легко модифицировать и расширять структуру БЗ по мере необходимости.

В реализованной модели знания представляются в виде узлов с меткой Element. Каждый такой узел обладает набором свойств, ключевыми из которых являются уникальный идентификатор (внутренний ID Neo4j), name (наименование или заголовок фрагмента знания) и description (детальное текстовое содержание). Иерархические отношения между узлами,

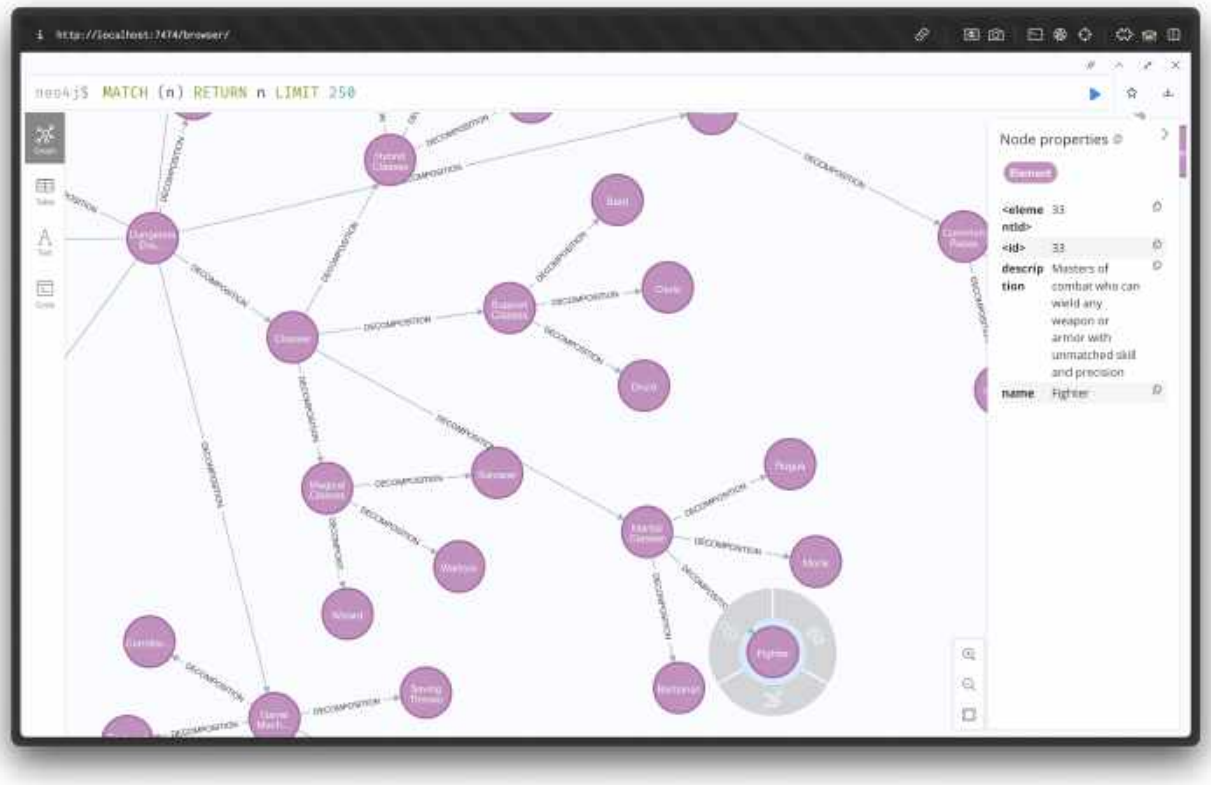


Рисунок 3.3 – Пример фрагмента графа знаний в Neo4j, иллюстрирующий узлы Element и связи :DECOMPOSITION для нескольких связанных понятий

отражающие декомпозицию или детализацию понятий, реализуются с помощью связей (Relationships) типа :DECOMPOSITION. Эти связи являются направленными, указывая от более общего (родительского) узла к более частному (дочернему). Например, в базе знаний по Dungeons Dragons, используемой для демонстрации, категория “Classes“ связана отношением :DECOMPOSITION с узлами “Martial Classes“ и “Magical Classes“, которые, в свою очередь, декомпозируются на конкретные классы персонажей, такие как “Fighter“ или “Wizard“, рисунок 3.3.

Первичная загрузка данных в базу знаний на текущем этапе разработки осуществляется путем выполнения заранее подготовленных Cypher-скриптов. Эти скрипты содержат команды для создания узлов и связей, формирующих начальную структуру и наполнение БЗ. Обновление и модификация БЗ также производятся посредством выполнения Cypher-запросов. В перспективе, для более удобного управления знаниями, предполагается разработка специализированного административного интерфейса.

3.3.2 Реализация хранилища контекстной информации

Для корректной работы итеративного алгоритма поиска и обеспечения осмысленности диалога с LLM-ассистентом необходимо эффективное управление контекстной информацией в рамках каждой сессии обработки запроса. В текущей MVP-реализации контекст сессии полностью инкапсулирован и хранится в оперативной памяти экземпляра класса Context на протяжении всего жизненного цикла обработки одного пользовательского запроса.

Такой подход выбран в силу его простоты и высокой производительности, так как доступ к данным в памяти осуществляется очень быстро, что важно для итеративного характера поискового процесса.

Хотя хранение контекста в памяти экземпляра класса Context является достаточным для текущих задач, в более сложных сценариях, предполагающих длительные или прерываемые диалоги, а также для кеширования, может потребоваться внедрение внешнего in-memory хранилища, такого как Redis[31]. Это позволило бы сохранять состояние диалога между несколькими независимыми обращениями пользователя или между различными сессиями.

3.4 Реализация ядра интеллектуальной обработки запросов

Центральным элементом разработанной системы является интеллектуальное ядро, отвечающее за обработку пользовательских запросов. В отличие от традиционных поисковых систем или чисто генеративных языковых моделей, данное ядро реализует итеративный, управляемый LLM, навигационный поиск по структурированной графовой базе знаний. Основная цель этого подхода – не просто найти релевантную информацию, но сделать это максимально прозрачно и объяснимо для пользователя, предоставляя возможность уточнения контекста через диалоговое взаимодействие, если это необходимо. Ключевым компонентом, инкапсулирующим всю логику этого сложного процесса, является класс Context, экземпляры которого управляют каждой сессией обработки запроса от начала и до конца.

3.4.1 Реализация алгоритма последовательного интеллектуального поиска

Основной алгоритм навигации по графу знаний и принятия решений выполняется итеративно. Каждая итерация представляет собой шаг “погружения“ в базу знаний до тех пор, пока не будет найден окончательный ответ или не будет достигнут установленный предел глубины поиска.

Процесс начинается с получения исходного запроса пользователя и максимального количества разрешенных итераций. При первом шаге, когда информация о предыдущих этапах отсутствует, система выполняет начальный поиск в базе знаний Neo4j, извлекая узлы верхнего уровня иерархии. На последующих итерациях, если языковая модель на предыдущем шаге рекомендовала определенные узлы для дальнейшего исследования, ядро извлекает следующий приоритетный узел из этого списка и выполняет более сфокусированный поиск. Этот поиск начинается от указанного узла и запрашивает его дочерние и связанные элементы, при этом исключая уже посещенные ранее узлы во избежание зацикливания.

Полученные из базы знаний узлы, представляющие собой фрагменты знаний, преобразуются в специальный текстовый формат. Этот формат, включающий идентификатор узла, его полный путь в иерархии, заголовок, описание и информацию о наличии дочерних элементов, передается в качестве контекста языковой модели. Затем формируется полный промпт для LLM, который также включает исходный запрос пользователя, историю предыдущих взаимодействий с LLM в текущей сессии и, возможно, комментарий LLM с предыдущего шага.

После отправки запроса к LLM и получения ответа в формате JSON[32], система анализирует его. На основе этого ответа принимается решение о дальнейших действиях. Если LLM указывает на конкретный идентификатор узла как на содержащий достаточную информацию для ответа, этот идентификатор сохраняется, и соответствующий путь из базы знаний добавляется в историю пройденных путей, после чего цикл поиска завершается. Если же LLM возвращает список идентификаторов узлов для дальнейшего исследования, эти узлы, после необходимой фильтрации, добавляются в очередь для обработки на следующих итерациях. Также сохраняются любые указания на вспомогательные знания и текстовый комментарий от LLM, объясняющий его текущее решение. На каждом шаге идентификаторы узлов, которые были представлены LLM для анализа, фиксируются как посещенные. Хотя текущая реализация ядра не инициирует прямых запросов к пользователю для уточнения, сам механизм итеративного анализа и возможность LLM указать в своем комментарии на нехватку информации создают основу для диалогового взаимодействия, которое может быть реализовано на уровне внешнего адаптера.

3.4.2 Интеграция с языковыми моделями (LLM)

Интеграция с языковыми моделями является фундаментальной для интеллектуального ядра, однако их роль строго определена: LLM выступают в качестве интеллектуальных навигаторов и анализаторов, а не как генераторы конечного текстового ответа. Технически взаимодействие с LLM

осуществляется через специализированный клиент, который отправляет запросы к OpenAI-совместимому API, используя заранее сконфигурированную модель и эндпоинт. Ключ API для доступа к сервису LLM безопасно извлекается из переменных окружения. При каждом обращении к LLM формируется структурированный массив сообщений, который включает системный промпт, историю предыдущих обменов в текущей сессии (для поддержания контекста анализа) и текущий пользовательский запрос вместе с предоставленными фрагментами знаний из базы.

Ключевую роль в управлении поведением LLM играет системный промпт. Он тщательно разработан для того, чтобы инструктировать LLM выполнять строго определенные задачи: анализировать запрос пользователя относительно предоставленных фрагментов знаний и возвращать ответ исключительно в виде структурированного JSON-объекта. Этот JSON должен содержать поля, указывающие на идентификатор узла с окончательным ответом (если таковой найден и LLM в нем уверен), идентификаторы вспомогательных узлов, список идентификаторов узлов для дальнейшего исследования и текстовый комментарий, объясняющий логику LLM. Особенно важной является инструкция, предписывающая LLM всегда стремиться к более глубокому анализу, если это может привести к более полному и точному ответу, и не предлагать окончательное решение при наличии лишь поверхностной или неполной информации. Таким образом, LLM не генерирует текст ответа самостоятельно, а предоставляет ядру системы “указания“ для дальнейшей навигации по базе знаний или для формирования ответа на основе уже найденного верифицированного фрагмента.

3.4.3 Формирование и объяснение ответов

После завершения итеративного поиска, ядро системы приступает к формированию окончательного ответа для пользователя. Этот процесс имеет две основные ветки логики в зависимости от исхода поисковой сессии.

В случае успешного нахождения релевантного знания, когда системой был определен идентификатор узла с достаточной информацией, из базы знаний Neo4j запрашивается полная информация об этом узле. Затем формируется специальный финальный промпт для LLM. В этот промпт передается исходный запрос пользователя, точное содержание найденного узла, опционально – содержание вспомогательных узлов, и, что очень важно, все накопленные комментарии LLM, которые описывают логику поиска на каждом предыдущем шаге. Системные инструкции для этого финального этапа строго предписывают дословно процитировать предоставленное содержание найденного узла без каких-либо домыслов или перефразирований. После цитаты LLM должен, на основе переданных ему комментариев о ходе поиска, кратко изложить этапы, которые привели к данному ответу.

Дополнительно, если у найденного узла в базе знаний имеются прямые дочерние узлы, информация о них также извлекается и может быть добавлена к ответу в раздел “Дополнительная информация“, предоставляя пользователю возможность для дальнейшего самостоятельного изучения темы.

Если же поиск завершился неудачно (например, был достигнут установленный лимит итераций без нахождения ответа), также используется финальный промпт для LLM. Однако в этом случае в качестве основного знания передается специальная информация о том, что ответ не найден, с указанием причины (например, исчерпание попыток). LLM, опираясь на накопленные комментарии о безуспешных попытках поиска, формирует объяснение, почему системе не удалось найти релевантный ответ.

Важнейшим аспектом является объяснимость ответа. Помимо текстового объяснения от LLM, система предоставляет пользователю информацию о пути навигации по базе знаний. Этот путь формируется на основе истории перемещений по графу знаний, зафиксированной во время поиска. Специальная утилита обрабатывает эту историю, создавая единый и логичный маршрут от корневого узла до найденного решения (или до точки, где поиск был остановлен). Этот путь может быть отображен пользователю через интерфейс адаптера, наглядно демонстрируя, как именно система пришла к предложенному ответу и какие разделы базы знаний были задействованы в процессе.

3.4.4 Реализация интеллектуальных механизмов автоматизации

Хотя разработанное интеллектуальное ядро спроектировано как универсальный компонент и не содержит логики, жестко привязанной к специфике “обработки обращений“, его архитектура и функциональность предоставляют мощные инструменты для автоматизации процессов именно в этой предметной области.

Например, если в результате поиска ядро успешно определяет идентификатор узла, содержащего решение типичной проблемы или ответ на часто задаваемый вопрос, система управления обращениями может автоматически предоставить этот ответ пользователю, инициировавшему обращение, без необходимости привлечения оператора. В случаях, когда LLM на начальных этапах поиска предлагает список узлов для дальнейшего исследования, эта информация может быть использована для интеллектуальной классификации и маршрутизации входящего обращения. В зависимости от тематики первых предложенных узлов, обращение может быть автоматически направлено в соответствующий отдел поддержки или передано экспертной группе с нужной специализацией.

Механизм итеративного поиска, в рамках которого LLM может указывать на нехватку информации через свои комментарии или предлагая специфические узлы для дальнейшего изучения, позволяет системе управления обращениями автоматически запрашивать у пользователя дополнительные сведения. Это помогает собрать всю необходимую информацию для точного определения проблемы до того, как обращение, если это потребуется, будет эскалировано на оператора.

Сам механизм эскалации также становится более интеллектуальным. Если ядро завершает свою работу, сигнализируя о невозможности найти релевантное знание в базе после установленного числа попыток, это служит четким триггером для передачи обращения оператору. При этом оператору предоставляется не просто исходный текст обращения, а вся накопленная в ходе автоматического поиска информация: исходный запрос, полная история комментариев LLM, описывающая предпринятые попытки анализа, и итоговый путь навигации по базе знаний. Такой обогащенный контекст значительно сокращает время, необходимое оператору для погружения в суть проблемы и поиска эффективного решения.

В перспективе, на базе этого универсального ядра могут быть реализованы и другие интеллектуальные функции, специфичные для системы обработки обращений. К ним можно отнести проактивное предложение нескольких вариантов решений, если однозначный ответ не найден, или использование обратной связи от пользователей для динамической корректировки релевантности узлов знаний и улучшения качества работы системы в целом.

3.5 Реализация пользовательских интерфейсов (Адаптеры)

Для взаимодействия конечных пользователей с разработанным интеллектуальным ядром необходимы специализированные программные компоненты – адаптеры. Основная задача адаптера заключается в обеспечении удобного и привычного для пользователя интерфейса для отправки запросов и получения ответов от системы, а также в трансляции этих взаимодействий в формат, понятный ядру, и обратно. Архитектура ядра изначально проектировалась с учетом необходимости легкой интеграции с различными типами таких адаптеров, будь то мессенджеры, веб-приложения или API-сервисы. Это достигается за счет четко определенной точки входа в ядро и стандартизированного протокола обмена данными с ним, что позволяет разрабатывать новые адаптеры с минимальными усилиями и без модификации самого ядра.

3.5.1 Реализация адаптера для Telegram-бота

В рамках текущей разработки в качестве основного демонстрационного пользовательского интерфейса был реализован адаптер в виде Telegram-бота. Выбор Telegram обусловлен его широкой популярностью, удобным API для разработчиков и возможностью быстрого прототипирования диалоговых систем.

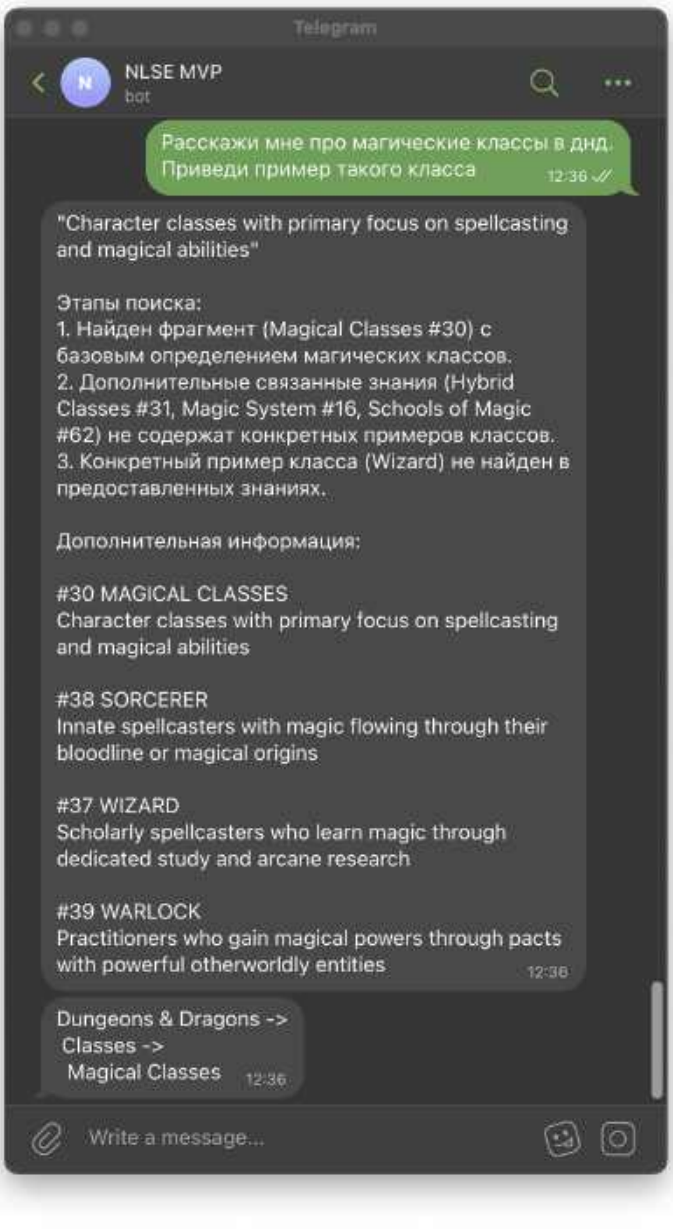


Рисунок 3.4 – Пример поиска по базе знаний через Telegarm-бот API

Логика работы Telegram-адаптера построена на последовательном взаимодействии с интеллектуальным ядром системы. При поступлении текстового сообщения от пользователя, адаптер инициирует новую сессию

обработки в ядре, передавая ему содержание пользовательского запроса. После этого адаптер запускает итеративный процесс поиска, последовательно запрашивая у ядра выполнение следующего шага анализа до тех пор, пока ядро не сообщит о нахождении окончательного ответа или о достижении установленного предела глубины поиска. По завершении этого цикла обработки, адаптер получает от ядра финальный, уже отформатированный для пользователя ответ, и передает его обратно в чат Telegram. Важным аспектом является также предоставление пользователю информации о процессе принятия решения: адаптер запрашивает у ядра метаданные о пути навигации по базе знаний, который привел к найденному ответу, и отображает эту информацию, обеспечивая прозрачность и объяснимость работы системы, рисунок 3.4.

Таким образом, Telegram-бот выступает в роли тонкого клиента, делегируя всю сложную логику обработки запроса и поиска информации интеллектуальному ядру. Это демонстрирует гибкость архитектуры и простоту интеграции ядра с различными каналами взаимодействия. В будущем аналогичные адаптеры могут быть разработаны для других платформ, например, для веб-интерфейса или голосовых ассистентов, используя тот же самый унифицированный API ядра.

3.5.2 Реализация web-приложения

Для обеспечения альтернативного, визуально насыщенного канала взаимодействия с интеллектуальным ядром, а также для предоставления инструментов администрирования, было разработано web-приложение. Оно построено на базе современного стека технологий, включающего React для создания пользовательского интерфейса, TypeScript для обеспечения строгой типизации и повышения надежности кода, и Vite в качестве инструмента сборки, обеспечивающего быструю разработку. Стилизация компонентов выполнена с использованием Tailwind CSS, а для готовых UI-компонентов (кнопки, карточки, формы и т.д.) была задействована библиотека shadcn/ui, что позволило быстро создать консистентный и адаптивный дизайн.

Архитектура и ключевые компоненты приложения включают компонентный подход. Управление состоянием, связанным с процессом обработки запроса (текущий запрос, статус, путь навигации, ответ, история запросов), осуществляется централизованно с помощью React Context API. Аналогичный подход используется для управления темой оформления, позволяя пользователю выбирать между светлой, темной и системной темами, с запоминанием выбора. Навигация между разделами приложения (Главная, История, Администрирование) реализована с использованием URL hash. Пользовательский интерфейс взаимодействия с системой состоит из нескольких ключевых страниц.

Главная страница выступает основной точкой взаимодействия. Здесь расположена форма для ввода запроса/обращения пользователя, рисунок 3.5.

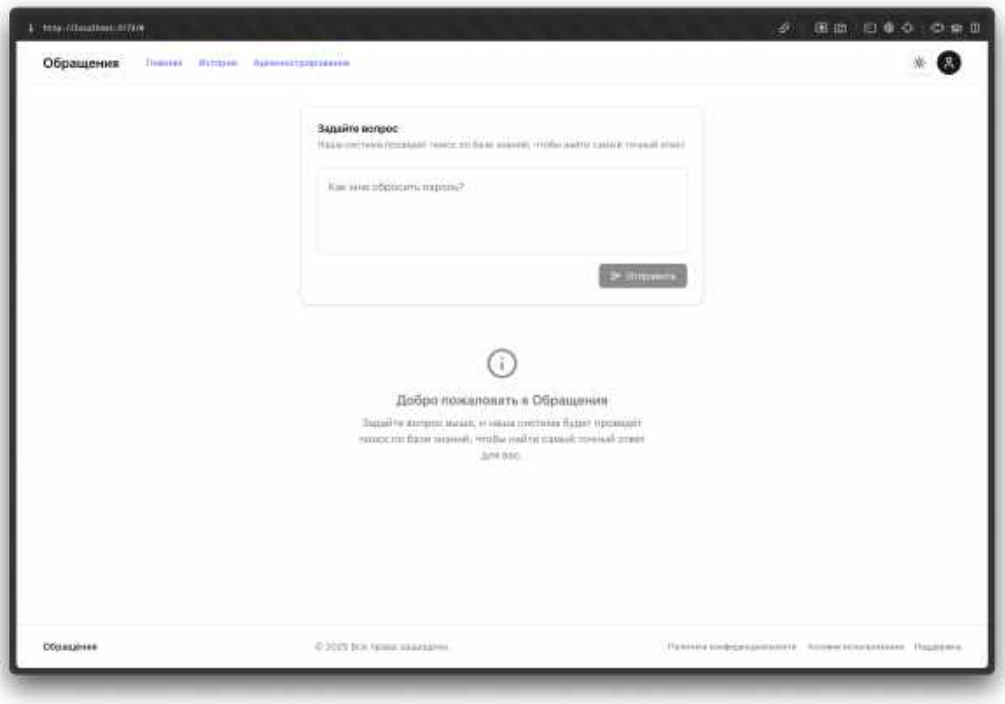


Рисунок 3.5 – Главный экран веб-приложения для ввода запроса

После отправки запроса на этой же странице динамически отображается процесс его обработки: поэтапно выводятся статусы поиска решения, рисунок 3.6 .

По мере выполнения каждого этапа интеллектуального поиска, система отображает список этих этапов и указывает, с какой степенью уверенности LLM (большая языковая модель) сделала выбор на каждом шаге, рисунок 3.7.

По завершении поиска, пользователю представляется результат: текстовый ответ на его изначальный вопрос, а также ссылки на источники знаний в графовой базе, на основе которых был сформирован ответ, рисунок 3.8.

Если пользователь не удовлетворен предложенным системой решением, он имеет возможность эскалировать обращение на оператора. В этом случае обращение помечается соответствующим образом в системе, рисунок 3.9.

Другим важным разделом является страница истории запросов, которая предоставляет пользователю доступ ко всем его предыдущим обращениям. Запросы отображаются в виде списка карточек, каждая из которых

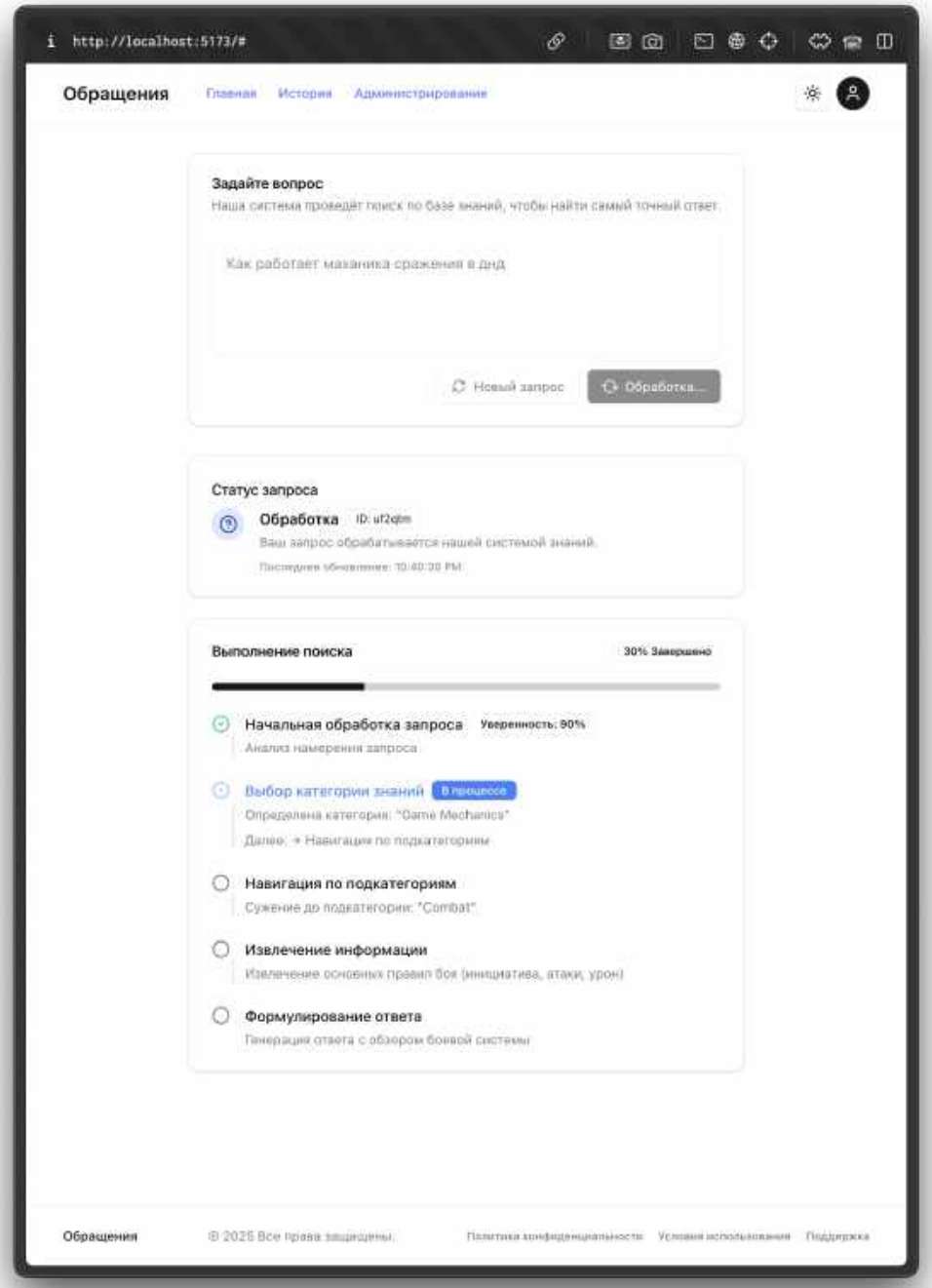


Рисунок 3.6 – Отображение процесса обработки обращения со статусами

содержит текст запроса, его финальный статус, временную метку, количество найденных источников и процент уверенности ответа. Предусмотрена функция очистки истории запросов. Пример страницы истории показан на рисунке 3.10.

Интерфейс администрирования предназначен для управления базой знаний и настройками системы. Этот раздел предоставляет более продви-

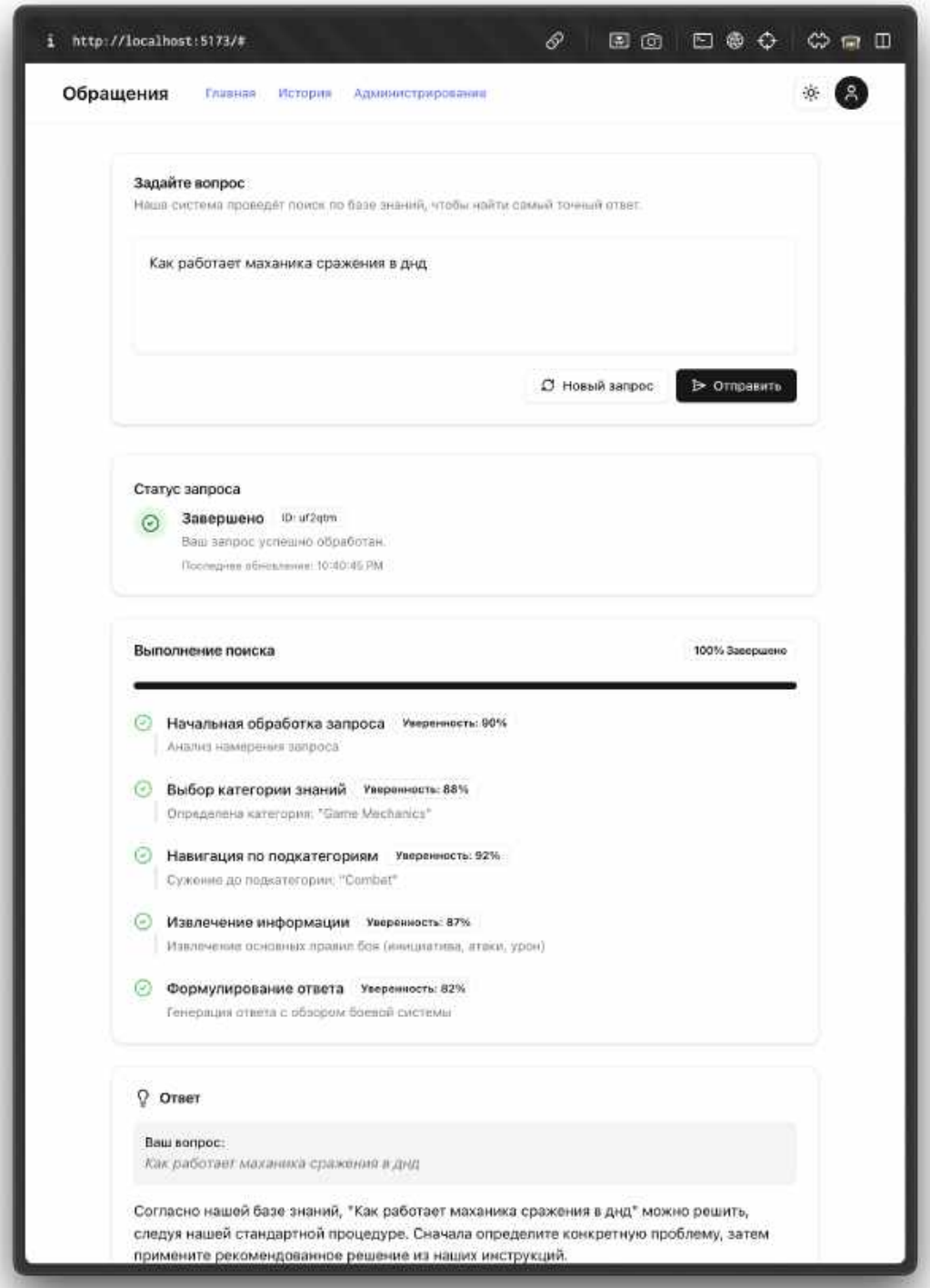


Рисунок 3.7 – Список выполненных этапов поиска с указанием уверенности LLM

нутые инструменты и разделен на несколько функциональных вкладок. Вкладка База знаний содержит интерактивную визуализацию графа знаний, позволяющую администратору наглядно представить структуру БЗ, связи между узлами, а также осуществлять поиск по графу. При выборе узла в

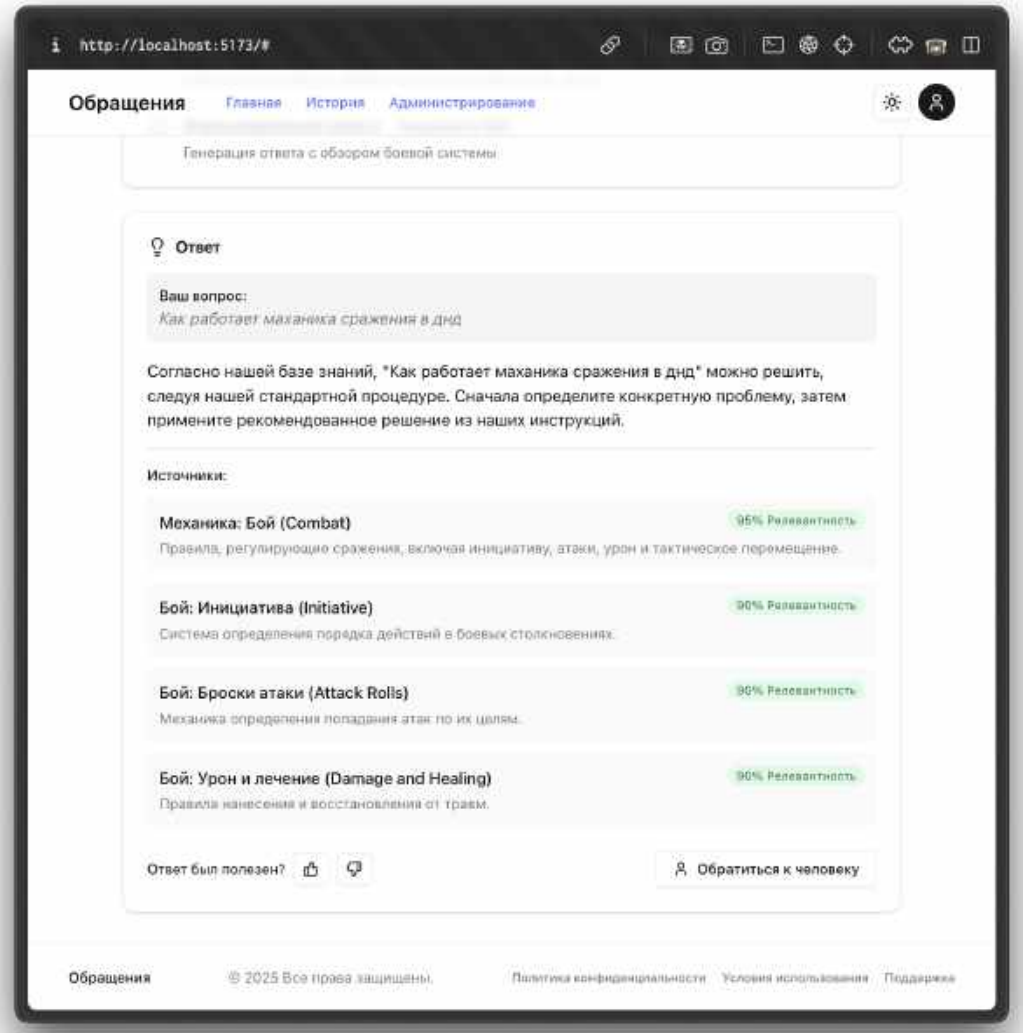


Рисунок 3.8 – Результат поиска решения с текстом ответа и ссылками на источники

графе, в отдельной панели отображаются его детали (название, содержимое, теги). Также на этой вкладке доступны базовые операции, такие как добавление или удаление фрагментов. Пример интерфейса управления графом знаний приведен на рисунке 3.11. Вкладка «Аналитика» предназначена для отображения статистики использования системы и производительности базы знаний, однако в текущей реализации содержит заглушку. Вкладка «Настройки» позволяет администратору конфигурировать ключевые параметры системы, такие как порог доверия для автоматического ответа и список email-адресов операторов для эскалации запросов. На странице администрирования также выводится общая статистика по базе знаний: количество фрагментов, связей, количество запросов и средняя уверенность ответов.

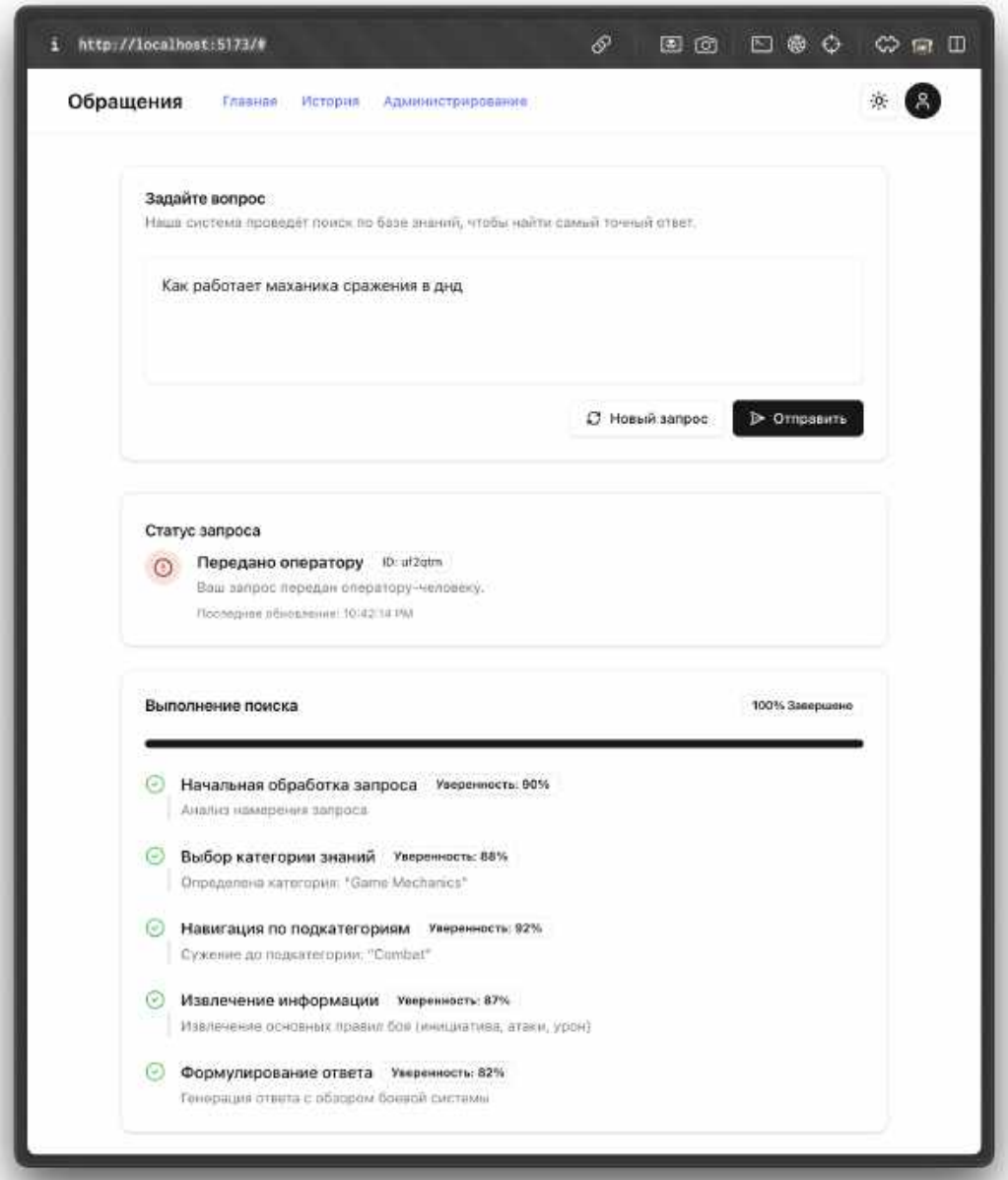


Рисунок 3.9 – Эскалация обращения на оператора

Таким образом, разработанное web-приложение является ещё одним каналом для обработки обращений помимо Telegram-бота, предоставляя богатые возможности для визуализации процесса поиска и администрирования системы. Использование React и TypeScript обеспечивает хорошую структурированность кода и потенциал для дальнейшего развития интерфейса.

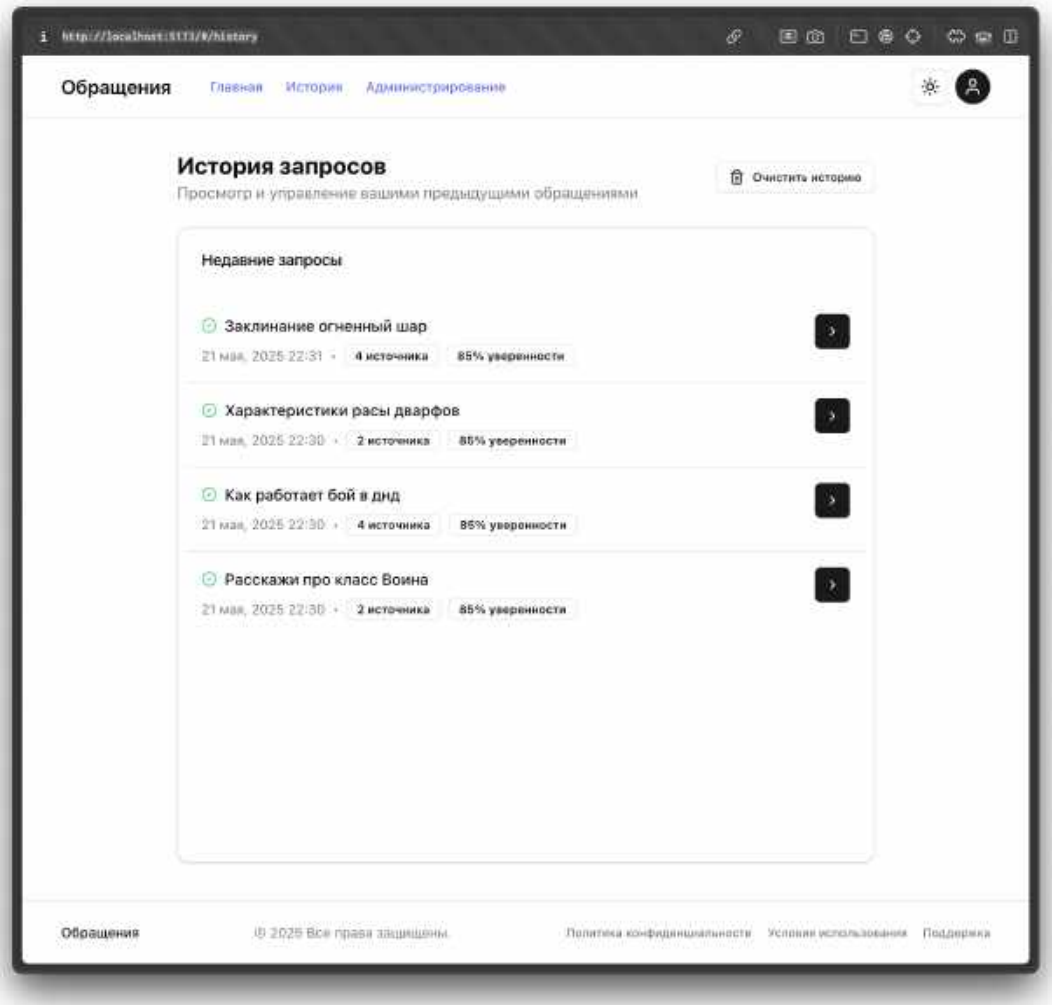


Рисунок 3.10 – Страница истории запросов в веб-приложении

3.6 Результаты разработки и демонстрация работы

В результате проделанной работы была создана интегрированная интеллектуальная система управления потоками пользовательских обращений. Система включает в себя универсальное ядро, разработанное на TypeScript с использованием среды Deno, которое реализует алгоритм последовательного интеллектуального поиска по иерархической базе знаний, хранящейся в графовой СУБД Neo4j. Ключевую роль в навигации и анализе играют большие языковые модели (LLM), взаимодействие с которыми осуществляется через OpenAI-совместимый API. Для взаимодействия с пользователями реализованы два адаптера: Telegram-бот для оперативного диалогового доступа и многофункциональное web-приложение на React, предоставляющее расширенные возможности визуализации, истории запросов и администрирования базы знаний. Система спроектирована с акцентом

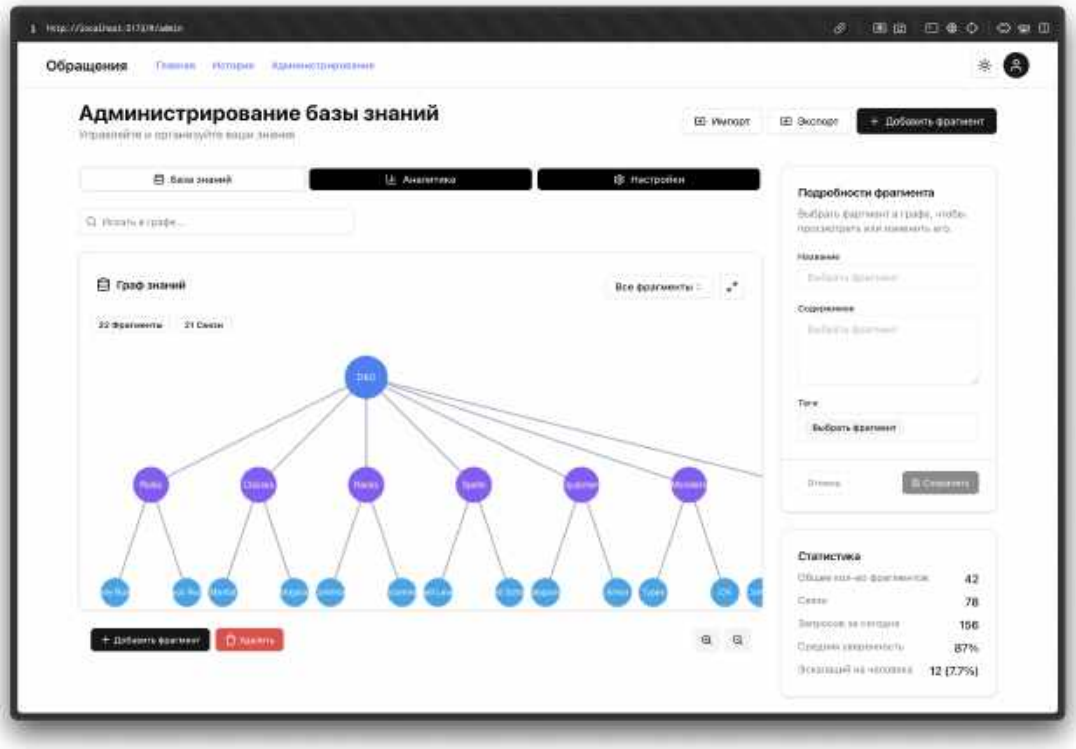


Рисунок 3.11 – Страница администрирования с отображением графа знаний

на объяснимость принимаемых решений, предоставляя пользователю не только ответ, но и путь его получения.

Функциональность и практическая применимость разработанной системы были продемонстрированы на ряде тестовых сценариев, охватывающих различные типы пользовательских запросов к базе знаний по тематике Dungeons & Dragons. В ходе демонстрации были показаны процессы приема запроса через пользовательские интерфейсы, пошаговая работа интеллектуального ядра по навигации в графе знаний с участием LLM, формирование структурированного ответа и его представление пользователю, включая визуализацию пути поиска. Примеры работы интерфейсов и логика принятия решений были подробно рассмотрены в предыдущих подразделах (см. рисунки 3.4, 3.5, 3.10, 3.11).

Хотя система является прототипом, первоначальная оценка ее потенциальной эффективности указывает на значительные перспективы. По предварительным расчетам, автоматизированная обработка типовых запросов с использованием разработанного ядра способна сократить среднее время нахождения ответа с нескольких десятков минут (при ручной обработке оператором) до 10-30 секунд, в зависимости от сложности запроса и глубины необходимого поиска. Ожидается, что система сможет корректно и полностью автономно разрешать до 60-70% стандартных обращений, при

условии, что информация четко структурирована в базе знаний. Это, в свою очередь, может привести к снижению нагрузки на операторов первой линии поддержки по данным категориям запросов, позволяя им сконцентрироваться на более сложных и нестандартных случаях. Важным качественным результатом является повышение прозрачности процесса поиска решения для конечного пользователя за счет механизма объяснения и визуализации пути навигации. Дальнейшее наполнение и структуризация базы знаний, а также тонкая настройка промптов для LLM, позволят улучшить эти показатели.

3.7 Вывод по разделу

В рамках данной главы была подробно описана практическая реализация интегрированной интеллектуальной системы управления потоками пользовательских обращений. Ключевыми аспектами реализации стали разработка универсального интеллектуального ядра на TypeScript/Deno, использующего графовую базу данных Neo4j для хранения иерархических знаний и большие языковые модели для навигации и анализа. Были успешно имплементированы механизмы итеративного поиска, формирования объяснимых ответов и управления контекстом сессии. Для взаимодействия с пользователем созданы два адаптера: Telegram-бот и web-приложение на React, демонстрирующие гибкость интеграции с ядром.

Реализованная система в полной мере соответствует проектным решениям, изложенным в предыдущих главах, и удовлетворяет основным функциональным требованиям, предъявляемым к минимально жизнеспособному продукту (MVP). Архитектура с четким разделением на ядро и адаптеры, использование унифицированного API для взаимодействия, а также применение современных и гибких технологий подтверждают успешное воплощение концепции интеллектуальной системы. Продемонстрированные сценарии работы подтверждают способность системы обрабатывать запросы, осуществлять навигацию по базе знаний и предоставлять структурированные, объяснимые ответы.

Созданный прототип обладает высоким потенциалом для дальнейшего развития. Модульная структура ядра, четко определенные интерфейсы взаимодействия и выбранный технологический стек (TypeScript, Deno, React, Neo4j) закладывают прочную основу для расширения функциональности, масштабирования и возможного перехода к микросервисной архитектуре. Система готова к дальнейшему наполнению базы знаний, интеграции с другими каналами взаимодействия, а также к внедрению более сложных интеллектуальных механизмов, таких как адаптивное обучение и проактивное предложение решений.

4 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ СИСТЕМЫ МОНИТОРИНГА И УПРАВЛЕНИЯ ПРОИЗВОДСТВЕННЫМИ ЛИНИЯМИ

4.1 Характеристика программного средства

Целью дипломного проекта является разработка интегрированной системы управления потоками пользовательских обращений.

Целью разработки системы интегрированной автоматизированной обработки запросов является оптимизация процессов обработки пользовательских обращений, сокращение времени ответа на запросы, повышение точности предоставляемой информации, а также снижение нагрузки на персонал за счет автоматизации рутинных операций и интеллектуального анализа поступающих вопросов.

Преимуществами данного программного средства являются универсальность архитектуры, возможность адаптации к различным предметным областям, структурированное хранение знаний, наличие механизма уточняющих вопросов, способность взаимодействия с внешними системами, а также масштабируемость и гибкость в настройке под специфические требования организации.

Данное программное средство будет использоваться в компании СП «Модсен» ООО для повышения качества обслуживания клиентов, ускорения обработки внутренних и внешних запросов, снижения операционных затрат и оптимизации использования человеческих ресурсов в службе поддержки.

Основные функции, которые выполняет программное средство:

– анализ пользовательских запросов и навигация по иерархической базе знаний;

– формирование уточняющих вопросов для определения точной потребности пользователя;

– предоставление релевантной информации из структурированного хранилища знаний;

– взаимодействие с внешними системами для получения актуальных данных;

– инициирование необходимых действий в зависимости от контекста запроса.

Экономическая оценка целесообразности инвестиций в разработку и использование программного средства осуществляется на основе расчета и оценки следующих показателей: чистый дисконтированный доход, рентабельность инвестиций и срок окупаемости инвестиций.

4.2 Расчет затрат на разработку и определения цены программного средства

Для разработки системы мониторинга и управления производственными линиями, которая создается для собственных нужд, компания-разработчик составила документ на разработку программного средства. В документе определены требования к разрабатываемому программному средству и его стоимость. Стоимость программного средства рассчитана на основе полных затрат на разработку и включает следующие статьи: основная заработная плата разработчиков, дополнительная заработная плата разработчиков, отчисления на социальные нужды, прочие расходы, общая сумма затрат на разработку.

4.2.1 Расчет затрат на основную заработную плату разработчикам

Первым этапом расчета затрат на разработку системы является расчет оплаты команды разработчиков. Расчет осуществляется исходя из состава и численности команды, размера месячной заработной платы каждого участника команды, а также трудоемкости работ, выполняемых при разработке программного средства отдельными исполнителями по формуле:

$$З_О = К_{ПР} \cdot \sum_{i=1}^{n} З_{Чi} \cdot t_i, \tag{4.1}$$

где $К_{ПР}$ – коэффициент премий (равный 1,5);

n – категории исполнителей, занятых разработкой программного средства;

$З_{Чi}$ – часовая заработная плата исполнителя i-й категории, р.;

t_i –трудоемкость работ, выполняемых исполнителем i-й категории, определяется исходя из сложности разработки программного обеспечения и объема выполняемых им функций, ч.

На 2025 год расчетная норма рабочего времени для пятидневной рабочей недели составляет 168 часов, 8 часов работы в день, среднемесячная расчетная норма рабочего времени – 21 день.

Разработкой программного средства занимался техник-программист. Часовая заработная плата исполнителя определялась путем деления его месячной заработной платы на количество рабочих часов в месяце. Расчет основной заработной платы представлен в таблице 4.1.

Таблица 4.1 – Расчет основной заработной платы

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч.	Итого, р.
Инженер-программист	4120	24,52	520	12750,4
Техник-программист	2360	14,04	410	5756,4
Итого				18506,8
Премия и иные стимулирующие выплаты (20%)				4287,66
Основная заработная плата разработчиков (Зо)				22794,46

4.2.2 Расчет затрат на дополнительную заработную плату разработчикам

Дополнительная заработная плата – это оплата за сверхурочный труд, различные трудовые успехи и надбавки за особые условия труда команды и включает выплаты, предусмотренные законодательством о труде, и определяется по нормативу в процентах (составляет 20%) к основной заработной плате по следующей формуле:

$$З_д = \frac{З_o \cdot Н_д}{100\%}, \tag{4.2}$$

где $Н_д$ – норматив дополнительной заработной платы(20 %);
$З_о$ – затраты на основную заработную плату, (р.);
Подставим значение в формулу (4.2) и вычислим $З_д$:

$$З_д = \frac{22794,46 \cdot 20\%}{100\%} = 4558,89 \text{ р.}$$

Согласно расчетам, затраты на дополнительную заработную плату разработчикам составит 4558,89 рублей.

4.2.3 Расчет отчислений на социальные нужды.

В расчете отчислений на социальные нужды отражаются обязательные отчисления по установленным законодательством тарифам в фонд социальной защиты населения, а также расходы предприятия на обязательное социальное медицинское страхование некоторых категорий работников в соответствии с законодательством. Расчет размера отчислений в фонд социальной защиты населения и на обязательное страхование определяется в соответствии с действующими законодательными актами Республики Беларусь и вычисляется по формуле:

$$\text{Р}_{\text{соц}} = \frac{(\text{З}_o + \text{З}_\text{д}) \cdot H_{\text{соц}}}{100\%}, \tag{4.3}$$

где $H_{\text{соц}}$ – норматив отчислений на социальные нужды, %.

Согласно законодательству Республики Беларусь, отчисления на социальные нужды составляют 34% в фонд социальной защиты и 0,6% на обязательное страхование. Подставим результаты вычислений в формулу (4.3) и вычислим $\text{Р}_{\text{соц}}$:

$$\text{Р}_{\text{соц}} = \frac{(22794,46 + 4558,89) \cdot (34\% + 0,6\%)}{100\%} = 9464,26 \text{ р.}$$

Согласно расчетам, размер отчислений в фонд социальной защиты и на обязательное страхование составляет 9464,26 рублей.

4.2.4 Расчет затрат на прочие расходы

Прочие расходы связаны с функционированием организации-разработчика в целом, например: затраты на аренду офисных помещений, отопление, освещение, амортизацию основных производственных фондов и так далее. При расчете данной статьи затрат учитывается норматив прочих затрат в целом по организации. В данном случае норматив прочих затрат равен 20%. Размер затрат на прочие расходы рассчитывается по формуле:

$$\text{Р}_{\text{пр}} = \frac{\text{З}_o \cdot \text{Н}_{\text{пз}}}{100\%}, \tag{4.4}$$

где $\text{Н}_{\text{пз}}$ – норматив прочих затрат в целом по организации, 20 %.

Подставим значение из выражения в формулу (4.4) и произведем расчет $\text{Р}_{\text{пр}}$:

$$\text{Р}_{\text{пр}} = \frac{22794,46 \cdot 20\%}{100\%} = 4558,89 \text{ р.}$$

Согласно расчетам, размер затрат на прочие расходы составляет 4558,89 рублей.

4.2.5 Расчет суммы затрат на разработку

Общая сумма затрат на разработку рассчитывается путем суммирования основной заработной платы, дополнительной заработной платы, отчислений на социальные нужды, прочих затрат. Формула расчета имеет следующий вид:

$$\text{З}_\text{р} = \text{З}_\text{о} + \text{З}_\text{д} + \text{Р}_{\text{соц}} + \text{Р}_{\text{пр}}, \tag{4.5}$$

Подставим результаты вычислений в формулу (4.5) и произведем расчет Зр:

$$З_р = 22794,46 + 4558,89 + 9464,26 + 4558,89 = 41376,5 \text{ р.}$$

Согласно расчетам, сумма затрат на разработку составляет 41 376,5 рублей

4.2.6 Результаты расчета цены на разработку программного средства

В данном подразделе были рассчитаны необходимые статьи для расчета затрат на разработку, а именно: основная заработная плата разработчиков, дополнительная заработная плата разработчиков, отчисления на социальные нужды, прочие расходы и всего затрат на разработку. Результаты расчетов представлены в таблице 4.2.

Таблица 4.2 – Результаты расчета цены на разработку программного средства

Наименование статьи затрат	Сумма, р.
Основная заработная плата разработчиков	18506,8
Дополнительная заработная плата разработчиков	4558,89
Отчисления на социальные нужды	9464,26
Прочие расходы	4558,89
Общая сумма инвестиций (затрат) на разработку	41376,5

4.3 Расчет результата от разработки и реализации программного средства

Экономия на заработной плате и начислениях на заработную плату сотрудников за счет снижения трудоемкости работ необходимо рассчитывать по формуле:

$$Э_{з.п.} = К_{пр} \cdot ({t_р}^{\text{без п.с.}} - {t_р}^{\text{с п.с.}}) \cdot Т_ч \cdot N_П \cdot (1 + \frac{Н_д}{100}) \cdot (1 + \frac{Н_{соц}}{100}), \qquad (4.6)$$

где $N_П$ – плановый объем работ, выполняемый сотрудником;

${t_р}^{\text{без п.с.}}, {t_р}^{\text{с п.с.}}$ – трудоемкость выполнения работы до и после внедрения программного средства, нормо-час;

$Т_ч$ – часовой оклад (часовая тарифная ставка) сотрудника, использующего программное средство;

$К_{пр}$ – коэффициент премий (по фактическим данным предприятия или в диапазоне 1,5-2);

$Н_д$ – норматив дополнительной заработной платы, (20%);

$Н_{соц}$ – ставка отчислений от заработной платы, включаемых в себестоимость, (34,6%).

Подставим результат вычисления в формулу (4.8) и произведем расчет $Э_{з.п.}$:

$$Э_{з.п.} = 1,2 \cdot (1 - 0,6) \cdot 16 \cdot 2016 \cdot (1 + \frac{20}{100}) \cdot (1 + \frac{34,6}{100}) = 25007,95 \text{ р.}$$

Экономия на заработной плате и начислениях на заработную плату в результате сокращения численности работников определяется по формуле:

$$Э^{п}_{з.п.} = \sum_{i=1}^{n} \Delta Ч_{i} \cdot З_{i} \cdot (1 + \frac{Н_{д}}{100}) \cdot (1 + \frac{Н_{соц}}{100}), \quad (4.7)$$

где n – категории работников, высвобождаемых в результате внедрения программного средства;

$\Delta Ч_{i}$ – численность работников i-й категории, высвобожденных после внедрения программного средства, чел.;

$З_{i}$ – годовая заработная плата высвобожденных работников i-й категории после внедрения программного средства, р.;

$Н_{д}$ – норматив дополнительной заработной платы, (20%);

$Н_{соц}$ – ставка отчислений от заработной платы, включаемых в себестоимость, (34,6%).

Экономия на заработной плате и начислениях на заработную плату в результате сокращения численности работников составляет 0 р., поскольку сотрудники не были уволены.

Экономия на материальных ресурсах рассчитывается по формуле:

$$Э_{м} = К_{т.р.} \cdot (Н_{р}{}^{\text{без п.с.}} - Н_{р}{}^{\text{с п.с.}}) \cdot Ц_{м} \cdot N_{П}, \quad (4.8)$$

где $К_{т.р.}$ – коэффициент транспортных расходов (по данным предприятия или 1,05-1,2);

$Н_{р}{}^{\text{без п.с.}}, Н_{р}{}^{\text{с п.с.}}$– норма расхода материальных ресурсов при выполнении работ сотрудниками до и после внедрения программного средства, нат. ед.;

$Ц_{м}$ – цена за единицу материального ресурса, р.;

$N_{П}$ – плановый объем работ, выполняемых сотрудником с использованием программного средства.

$$Э_{м} = 1,05 \cdot (6,1 - 5,9) \cdot 12 \cdot 120 = 403,2 \text{ р.}$$

Экономия на материальных ресурсах равна 403,2 рубля.

Экономическим эффектом при использовании программного средства является прирост чистой прибыли, полученной за счет экономии на текущих затратах предприятия, который рассчитывается по формуле:

$$\Delta П_{ч} = (Э_{тек} - \Delta З^{п.с.}_{тек}) \cdot (1 - \frac{Н_{п}}{100}), \quad (4.9)$$

где $Э_{тек}$ – экономия на текущих затратах при использовании программного средства, р.;

$\Delta З^{п.с.}_{тек}$ – прирост текущих затрат, связанных с использованием программного средства (затраты на сопровождение программного средства, затраты на интернет-трафик и т.п.), р;

$Н_{п}$ – ставка налога на прибыль согласно действующему законодательству 20%.

$$\Delta П_{ч} = ((25007,95 + 403,2) - 0) \cdot (1 - \frac{20\%}{100}) = 25410,34 \text{ р}$$

4.4 Расчет показателей экономической эффективности разработки и использования программного средства в организации

Оценка экономической эффективности разработки и использования программного средства для собственных нужд зависит от результата сравнения затрат на его разработку (модернизацию, совершенствование (и полученного экономического эффекта (годового прироста чистой прибыли).

Так как сумма инвестиций (затраты) окупятся более чем за год, то для организации рассчитывается несколько показателей экономической эффективности. Для приведения доходов и затрат к настоящему моменту времени определяется коэффициент дисконтирования по формуле:

$$\alpha_t = \frac{1}{(1+d)^{t-t_p}}, \quad (4.10)$$

где d – требуемая норма дисконта, которая по своему смыслу соответствует устанавливаемому инвестором желаемому уровню рентабельности инвестиций, доли единицы;

t – порядковый номер года, доходы и затраты которого приводятся к расчетному году;

t_p – расчетный год, к которому приводятся доходы и инвестиционные затраты ($t_p = 1$).

Норму дисконта принимаем равным ставке рефинансирования Национального банка Республики Беларусь – 9,5%. Расчетный период составит четыре года. Таким образом, коэффициенты дисконтирования за каждый год составляют:

$$\alpha_1 = \frac{1}{(1+0,095)^{1-1}} = 1,$$

$$\alpha_2 = \frac{1}{(1+0,095)^{2-1}} = 0,91,$$

$$\alpha_3 = \frac{1}{(1+0,095)^{3-1}} = 0,83,$$

$$\alpha_4 = \frac{1}{(1+0,095)^{4-1}} = 0,76.$$

В течение первого года осуществляется разработка приложения, поэтому в первый год экономический эффект будет меньше планируемого. Для того, чтобы учесть этот факт, необходимо выяснить, сколько времени будет затрачено на разработку приложения.

Так как работа команды разработчиков осуществляется поэтапно, то затраченное время будет равно сумме трудоемкости работ команды, и составит 900 часов.

Расчет показателей эффективности инвестиций осуществляется в табличной форме (таблица 4.3).

В данном случае дисконтированный эффект нарастающим итогом превысит дисконтированные инвестиции на второй год. Дисконтированный срок окупаемости рассчитывается по формуле:

$$\text{Т}_{\text{ок}}(\text{РР}) = \frac{\sum_{t=1}^{n} \text{З}_t}{\frac{1}{n} \cdot \sum_{t=1}^{n} \Delta\text{П}_{\text{ч}t}} \tag{4.11}$$

Таким образом, дисконтированный срок окупаемости равен

$$\text{Т}_{\text{ок}}(\text{РР}) = \frac{31452,9}{\frac{1}{4} \cdot (9352,74 + 16895,28 + 16895,28 + 16895,28)} = 2,1 \text{ года.}$$

Индекс доходности инвестиций рассчитывается по формуле:

$$\text{ИД}_{PI} = \frac{\sum_{t=1}^{n} \Delta\text{П}_{\text{ч}i} \cdot \alpha_i}{\sum_{t=1}^{n} \cdot \text{З}_i \cdot \alpha_i} \tag{4.12}$$

Таким образом, индекс доходности инвестиций равен:

$$\text{ИД}_{PI} = \frac{9352,74 + 15374,70 + 14023,08 + 12840,41}{31452,9} = 1,64$$

4.5 Вывод

Разработка системы интегрированной системы управления потоками пользовательских обращений направленна на повышение надежности и отказоустойчивости IT-инфраструктуры компании «Модсен» ООО. Внедрение

Таблица 4.3 – Расчет эффективности инвестиций (затрат) в реализацию проектного решения

Показатель	Значение расчетного периода по годам			
	1-й год	2-й год	3-й год	4-й год
1. Прирост чистой прибыли, р.	13688,31	25410,34	25410,34	25410,34
2. Дисконтированный результат, р.	13688,31	23123,40	21090,58	19311,85
3. Инвестиции (затраты) в реализацию проектного решения, р.	41376,5	0	0	0
4. Дисконтированные инвестиции, р.	41376,5	0	0	0
5. Чистый дисконтированный доход по годам, р.	-27688,19	23123,40	21090,58	13376,82
6. Чистый дисконтированный доход нарастающим итогом, р.	-27688,19	-4564,79	16525,79	2992,61
7. Коэффициент дисконтиро¬вания, доли единицы	1	0,91	0,83	0,76

данного программного средства позволит сократить трудозатраты на отбраковку продукции, мониторинг состояния оборудования, а также выявления причин аварийных ситуаций, что в совокупности повысит эффективность производства.

Расчет инвестиций в разработку показал, что основная заработная плата разработчиков составит 22794,46 руб., а общая сумма затрат с учетом дополнительных выплат, отчислений на социальные нужды и прочих расходов достигнет 41376,5 руб. Экономический эффект от использования системы, рассчитанный на основе экономии текущих затрат и прироста чистой прибыли, составил 25410,34 руб., что подтверждает целесообразность инвестиций.

По итогам экономического обоснования были получены следующие результаты:

1. Общая стоимость затрат на разработку системы мониторинга и управления производственными линиями составила 41376,5 рублей.

2. Прирост чистой прибыли от внедрения программного средства достиг 25410,34 рублей.

3. Инвестиции окупятся за 2,1 года (примерно 675,25 дней), а индекс доходности инвестиций составил 1.64, что свидетельствует об эффективности проекта.

В результате экономического обоснования разработки системы мониторинга и управления производственными линиями, видно, что разработка данного программного средства является эффективной и вложение инвестиций в разработку целесообразно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была успешно решена комплексная задача анализа, проектирования и разработки интегрированной системы управления потоками пользовательских обращений. Основной целью являлось создание интеллектуального решения, способного повысить эффективность обработки запросов, снизить операционные издержки и улучшить качество взаимодействия с пользователями за счет автоматизации и предоставления прозрачных, объяснимых ответов.

Был проведён анализ существующих подходов и коммерческих решений выявил актуальную потребность в системах, которые не просто автоматизируют рутинные операции, но и обеспечивают глубокую интеграцию с верифицированными базами знаний, минимизируя риски, связанные с неконтролируемой генерацией контента большими языковыми моделями. В ответ на эти вызовы была предложена и реализована концепция системы, где LLM выступает в роли интеллектуального навигатора по иерархической базе знаний, а не прямого генератора ответов.

Ключевым результатом проделанной работы является создание программного прототипа. В рамках этой работы:

1. Разработанно универсальное ядро системы на TypeScript с использованием среды выполнения Deno. Это ядро реализует алгоритм последовательного интеллектуального поиска по иерархической базе знаний, хранящейся в графовой СУБД Neo4j, и управляет взаимодействием с LLM через OpenAI-совместимый API.

2. Разработаны механизмы итеративного поиска, формирования структурированных и объяснимых ответов с указанием пути их получения, а также управления контекстом пользовательской сессии.

3. Продемонстрированны два пользовательских интерфейса (адаптера): Telegram-бот для оперативного диалогового доступа и многофункциональное web-приложение на React, предоставляющее расширенные возможности визуализации процесса поиска, истории запросов и администрирования базы знаний.

Продемонстрированные сценарии работы подтвердили способность системы эффективно обрабатывать запросы, осуществлять навигацию по базе знаний и предоставлять пользователю не только релевантный ответ, но и информацию о процессе его получения. Это повышает доверие к системе и обеспечивает прозрачность ее функционирования. Предварительные оценки указывают на значительный потенциал разработанного подхода в сокращении времени обработки типовых запросов (потенциально с десятков минут до 10-30 секунд).

Созданный прототип обладает высоким потенциалом для дальнейшего развития. Модульная структура ядра, четко определенные интерфейсы вза-

имодействия и выбранный технологический стек (TypeScript, Deno, React, Neo4j) закладывают прочную основу для расширения функциональности, масштабирования и возможного перехода к микросервисной архитектуре. Перспективными направлениями являются дальнейшее наполнение базы знаний, тонкая настройка промптов для LLM, интеграция с другими каналами взаимодействия, а также внедрение более сложных интеллектуальных механизмов, таких как адаптивное обучение и проактивное предложение решений.

Таким образом, дипломный проект в полной мере достиг поставленных целей, продемонстрировав жизнеспособность и эффективность предложенной концепции. Разработанные решения и программный прототип представляют собой значимый шаг к созданию интеллектуальных систем поддержки нового поколения, способных трансформировать процессы взаимодействия с пользователями в различных предметных областях, обеспечивая высокую скорость, точность и прозрачность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Gladly Research, Customer Service Expectations Survey, 2022. [Электронный ресурс]. — Режим доступа: https://www.gladly.ai/resources/customer-service-reports-guides/driving-customer-loyalty/2020-customer-expectations-report/. — Дата доступа: 27.03.2025.

[2] Wei, Jason. Emergent abilities of large language models / Jason Wei, Wang. — 2022.

[3] Sunil Ramlochan, The Black Box Problem: Opaque Inner Workings of Large Language Models, 2023. [Электронный ресурс]. — Режим доступа: https://promptengineering.org/the-black-box-problem-opaque-inner-workings-of-large-language-models/. — Дата доступа: 27.03.2025.

[4] Zendesk: The Complete Customer Service Solution [Электронный ресурс]. — Режим доступа: https://www.zendesk.com/. — Дата доступа: 27.03.2025.

[5] SentiSum | AI-Powered Customer Experience Analytics [Электронный ресурс]. — Режим доступа: https://www.sentisum.com/. — Дата доступа: 27.03.2025.

[6] Labelf AI: Elevate your Customer Interactions [Электронный ресурс]. — Режим доступа: https://www.labelf.ai/. — Дата доступа: 27.03.2025.

[7] IMB Watson. [Электронный ресурс]. — Режим доступа: https://www.ibm.com/watson. — Дата доступа: 27.03.2025.

[8] Watson for Oncology. [Электронный ресурс]. — Режим доступа: https://habr.com/ru/companies/ibm/articles/250903/. — Дата доступа: 27.03.2025.

[9] SAP Datasphere Knowledge Graph. [Электронный ресурс]. — Режим доступа: https://www.sap.com/products/data-cloud/datasphere/features.html. — Дата доступа: 27.03.2025.

[10] arXiv, Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering. [Электронный ресурс]. — Режим доступа: https://arxiv.org/abs/2404.17723. — Дата доступа: 27.03.2025.

[11] Graph RAG: как RAG, только с графом знаний [Электронный ресурс]. — Режим доступа: https://blogs.epsilonmetrics.ru/graph-rag-kak-rag-tolko-c-grafom-znanij/. — Дата доступа: 27.03.2025.

[12] Не только векторные БД: графовый RAG для LLM и агентского ИИ [Электронный ресурс]. — Режим доступа: https://bigdataschool.ru/blog/news/machine-learning/graphrag-in-agentic-ai-systems.html. — Дата доступа: 27.03.2025.

[13] Измерение процесса управления инцидентами и запросами пользователей [Электронный ресурс]. — Режим доступа: https://edu.cleverics.ru/useful/stati-konsultantov-cleverics/589-incident-management-measurement. — Дата доступа: 27.03.2025.

[14] The WebSocket Protocol. [Электронный ресурс]. — Режим доступа: https://datatracker.ietf.org/doc/html/rfc6455. — Дата доступа: 27.03.2025.

[15] Alhir, Sinan Si. Learning UMLв / Sinan Si. Alhir. — New-York : O'Reilly Media, 2003. — 256 pp.

[16] Usecase диаграмма [Электронный ресурс]. — Режим доступа: https://habr.com/ru/articles/699522/. — Дата доступа: 27.03.2025.

[17] Hohpe, Gregor. Enterprise integration patterns: Designing, building, and deploying messaging solutions / Gregor Hohpe, Bobby Woolf. — Boston : Addison-Wesley Professional, 2003.

[18] Stopford, Ben. Designing Event-Driven Systems / Ben Stopford. — Sebastopol, CA : O'Reilly Media, Inc., 2018.

[19] Vernon, Gregory. Implementing Domain-Driven Design (CQRS) / Gregory Vernon. — Addison-Wesley Professional, 2016. — 576 P.

[20] Grafana: The open and composable observability platform | Grafana Labs [Электронный ресурс]. — Режим доступа: https://grafana.com/. — Дата доступа: 27.03.2025.

[21] Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / Robert C. Martin. — Prentice Hall, 2017. — 432 P.

[22] TypeScript [Электронный ресурс]. — Режим доступа: https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html. — Дата доступа: 27.03.2025.

[23] OpenAI API Documentation. [Электронный ресурс]. — Режим доступа: https://platform.openai.com/docs/api-reference. — Дата доступа: 27.03.2025.

[24] DeepSeek. DeepSeek API Documentation. [Электронный ресурс]. — 2025.

[25] GrammY. The Telegram Bot Framework. [Электронный ресурс]. — Режим доступа: https://grammy.dev/ref/. — Дата доступа: 27.03.2025.

[26] Tailwind CSS. [Электронный ресурс]. — Режим доступа: https://tailwindcss.com/. — Дата доступа: 27.03.2025.

[27] Shadcn UI [Электронный ресурс]. — Режим доступа: https://ui.shadcn.com/docs. — Дата доступа: 27.03.2025.

[28] Newman, Sam. Building Microservices: Designing Fine-Grained Systems / Sam Newman. — Sebastopol, CA : O'Reilly Media, 2015. — 280 pp.

[29] Apache Kafka [Электронный ресурс]. — Режим доступа: https://kafka.apache.org/. — Дата доступа: 27.03.2025.

[30] Cypher Neo4j [Электронный ресурс]. — Режим доступа: https://neo4j.com/docs/cypher-manual/current/introduction/. — Дата доступа: 27.03.2025.

[31] Redis - The Real-time Data Platform [Электронный ресурс]. — Режим доступа: https://redis.io/. — Дата доступа: 27.03.2025.

[32] Json format [Электронный ресурс]. — Режим доступа: https://ecma-international.org/publications-and-standards/standards/ecma-404/. — Дата доступа: 27.03.2025.

[33] Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов / В. Г. Горовой, А. А. Горюшкин, А. В. Грицай, В. А. Пархименко. — Минск : БГУИР, 2021. — 107 с.

[34] Prometheus - Monitoring system time series database [Электронный ресурс]. — Режим доступа: https://prometheus.io/. — Дата доступа: 27.03.2025.

[35] Доманов, А. Т. Стандарт предприятия. Дипломные проекты (работы). Общие требования. СТП 01-2017 / А. Т. Доманов, Н. И. Сорока. — Минск : БГУИР, 2017. — 169 с.

Обозначение	*Наименование*	*Дополнительные сведения*
	Текстовые документы	
БГУИР ДП 1-40 03 01 032 ПЗ	Пояснительная записка	79 с.
	Отзыв руководителя	
	Рецензия	
	Отчет о проверке дипломного	
	проекта на заимствования	
	Графический материал	
ГУИР.121703.001 ПЛ	Диаграмма итеративного	Формат А1
	алгоритма поиска	
ГУИР.121703.002 ПЛ	Диаграмма вариантов	Формат А1
	использования	
ГУИР.121703.003 ПЛ	Диаграмма обработки запроса	Формат А1
ГУИР.121703.004 ПЛ	Диаграмма архитектуры системы	Формат А1
ГУИР.121703.005 ПЛ	Диаграмма процесса обработки	Формат А1
	обращения	
ГУИР.121703.006 ПЛ	Диаграмма пользовательского	Формат А1
	интерфейса	
ГУИР.121703.007 ПЛ	Диаграмма последовательности	Формат А1
	обработки обращений	
	Электронный носитель информации с программным обеспечением и материалами	CD-R

					БГУИР ДП 1- 40 03 01 032 Д1

Изм.	*Л.*	*№ докум.*	*Подп.*	*Дата*	*Интегрированная система управления потоками пользовательских обращений*	*Лит*	*Лист*	*Листов*
Разраб.		*Левков*				*Т*	*79*	*79*
Пров.		*Ерофеев*						
Т.контр.		*Ерофеев*			*Ведомость документов*	*Кафедра ИИТ гр. 121703*		
Н.контр.		*Захаров*						
Утв.		*Шункевич*						